# Haskell Programming from first principles

## Chapter 1 - All You Need is Lambda

## Chapter 2 - Hello, Haskell!

- The `($)` operator evaluates everything to its right first and can thereby be used to delay function application.
- You can use *sectioning* to partially apply an infix function: `(+2) 2`
- Sectioning for subtraction only works when it is the first argument: `(1 -) x`. To partially apply subtraction the other way, use `(subtract 2) 3`

## Chapter 3 - Strings

- The `::` symbol means "has the type" in Haskell.

## Chapter 4 - Basic Datatypes

- Float and Double work for calculations, but Scientific ( `stack install scientific` ) is better for a lot of cases. You may also use `Data.Fixed` to avoid floating point math. There are better packages for handling money.

## Chapter 5 - Types

- All functions are curried and the function constructor `(->)` is right associative. Adding parentheses to make it explicit:
  - `f :: a -> a -> a` is equivalent to `f :: a -> (a -> a)`
  - `map :: (a -> b) -> [a] -> [b]` is `map :: (a -> b) -> ([a] -> [b])`
- A function of two arguments really just takes one argument and returns a function from one argument to a result.
- Function definition `(->)` is a type constructor and is right associative, but function application `f a` is left associative.

- An *uncurried* version of `(+)` would take a tuple of two values and add them together, where our *curried* version takes an argument and returns a function that takes another argument and returns a value. Some older functional languages use product types like tuples to express multiple arguments.
- Remember: `\i b -> i + (nonsense b)` is `\i -> \b -> i + (nonsense b)`

```
-- curry and uncurry are already defined in Prelude, so we are shadowing here
Prelude> curry f a b = f (a, b)
Prelude> :t curry
curry :: ((t1, t2) -> t) -> t1 -> t2 -> t
Prelude> uncurry f (a, b) = f a b
Prelude> :t uncurry
uncurry :: (t1 -> t2 -> t) -> (t1, t2) -> t

Prelude> :t fst
fst :: (t, b) -> t
Prelude> :t curry fst
curry fst :: t -> b -> t
Prelude> fst (1, 2)
1
Prelude> curry fst 1 2
1

Prelude> :t (+)
(+) :: Num a => a -> a -> a
Prelude> (+) 1 2
3
Prelude> uncurry (+) (1, 2)
3
```

- *Sectioning* is the partial application of infix operators and has a special syntax that allows you to partially apply the first or second argument.

```
Prelude> elem 9 [1..10]
True
Prelude> 9 `elem` [1..10]
True
Prelude> c = (`elem` [1..10])
Prelude> c 9
True
Prelude> c 25
False
```

- If a variable could be *anything* (parametric polymorphism), then there's little that can be done to it, because it has no specific methods. If it can be *some* types (constrained or ad hoc polymorphism with type classes), then it has some methods. If it is a concrete type, you lose the type flexibility but due to the additive nature of inheritance you gain more potential methods.
- `Num` is called a superclass and `Integral` and `Int` are subclasses. In Haskell, a subclass cannot override methods of the superclass. Inheritance extends downwards. A member of `Num` has `Num` methods. A member of `Integral` has `Num` and `Integral` methods. And a concrete `Int` has `Num`, `Integral`, and `Int` methods.
- Most numeric literals in Haskell are only constrained to `Num`, but if you add a decimal or divide with `(/)` you might get constrained to `Fractional`.
- A function is polymorphic when its parameters are polymorphic. *Parametricity* means the behavior of a function with respect to the types of its parametrically polymorphic arguments is uniform and the behavior *cannot* change just because it was applied to an argument of a different type.
- The `fromIntegral` number takes an `Integral` and returns a `Num`. It is useful when working with some of the Prelude functions that return `Int`.

## Chapter 6 - Type Classes

- A declaration of a type defines how that type is created, and a declaration of a type class defines how a set of types are *consumed* or used in computations. A type class is like an interface in other languages and are a means of ad hoc polymorphism. A type has an instance of a type class, meaning there is code that defines how the values and functions from that type class work for that type.

- Keep your type class instances for a type in the same file as that type.

```haskell
data DayOfWeek =
  Mon | Tue | Wed | Thu | Fri | Sat | Sun

data Date =
  Date DayOfWeek Int

instance Eq DayOfWeek where
  (==) Mon Mon = True
  (==) Tue Tue = True
  (==) Wed Wed = True
  (==) Thu Thu = True
  (==) Fri Fri = True
  (==) Sat Sat = True
  (==) Sun Sun = True
  (==) _ _     = False

instance Eq Date where
  (==) (Date weekday dayOfMonth)
       (Date weekday' dayOfMonth') =
    weekday == weekday'
    && dayOfMonth == dayOfMonth'
```

- `:set -Wall` in a repl or `-Wall` in build configuration will cause GHC to let us know when we're not handling all cases.
- With some constraints, we can derive `Eq`, `Ord`, `Enum`, `Bounded`, `Read`, and `Show`.

## Chapter 7 - More Functional Patterns

- `newtype` is different from `data` in that it permits only one constructor and only one field. We will use it in the following pattern matching example:

```haskell
-- registerUser.hs
module RegisteredUser where

newtype Username =
  Username String

newtype AccountNumber =
  AccountNumber Integer

data User =
    UnregisteredUser
  | RegisteredUser Username AccountNumber

printUser :: User -> IO ()
printUser UnregisteredUser =
  putStrLn "UnregisteredUser"

printUser (RegisteredUser
            (Username name)
            (AccountNumber accNum)) =
  putStrLn $ name ++ " " ++ show accNum
```

- In ghci, `:browse ModuleName` will list the type signatures and functions we load from a module.
- Guard syntax allows us to write compact functions for two or more possible outcomes. Guards always evaluate sequentially. You can also use `where` declarations within guard blocks.

```haskell
myAbs :: Integer -> Integer
myAbs x
  | x < 0     = (-x)
  | otherwise = x

avgGrade :: (Fractional a, Ord a) => a -> Char
avgGrade x
  | y >= 0.9  = 'A'
  | y >= 0.8  = 'B'
  | y >= 0.7  = 'C'
  | y >= 0.59 = 'D'
  | y <  0.59 = 'F'
  where y = x / 100
```

- `(f . g) x = f (g x)`. `.` or `(.)` is the composition operator. You can think of the composition operator as a way of pipelining data through multiple functions. The composition operator has a precedence of 9, versus function application being 10, so you will sometimes see it used with `$` like `negate . sum $ xs`. You could also say `(negate . sum) xs`. The functions in composition are applied from right to left.
- "Point-free style" is a style of composing functions without specifying their arguments. The "point" in "point-free" refers to the arguments. We add "points" ( `.` ) to drop points (arguments). It helps the reader focus on the functions instead of the data. `f = negate . sum`.

## Chapter 8 - Recursion

- Recursion is defining a function in terms of itself via self-referential expressions. It is a means of expressing code that must take an *indefinite* number of steps to return a result. The data we are processing decides when we are done computing.
- The lambda calculus doesn't provide an obvious means of recursion due to the anonymity of expressions, but recursive functions are essential to Turing completeness. The Y combinator or fixed-point combinator allows us to write recursive functions in the lambda calculus and is the basis for the recursion in Haskell.
- A recursive function needs a *base case* that stops the self-application or else it will run forever.

```
applyTimes :: (Eq a, Num a) => a -> (b -> b) -> b -> b
applyTimes 0 f b = b
applyTimes n f b = f . applyTimes (n-1) f $ b -- f (applyTimes (n-1) f b)
```

- *bottom* refers to computations that do not successfully result in a value. The two main varieties are computations that fail with an error or those that fail to terminate. A partial function would be the first case.
- Type `type` keyword is used to declare a type synonym or type alias.
- Recall that multiplication is repeated addition and division is repeated subtraction. The quotient is how many times you can subtract the denominator from the numerator for a positive result.

```haskell
-- this is partial and doesn't handle divisors of 0 or less
dividedBy :: Integral a => a -> a -> (a, a)
dividedBy num denom = go num denom 0
  where go n   d count
          | n < d = (count, n)
          | otherwise =
             go (n - d) d (count + 1)
```

- Here's another example with recursion:

```haskell
import Data.List (intersperse)

digitToWord :: Int -> String
digitToWord n
 | n == 0 = "zero"
 | n == 1 = "one"
 | n == 2 = "two"
 | n == 3 = "three"
 | n == 4 = "four"
 | n == 5 = "five"
 | n == 6 = "six"
 | n == 7 = "seven"
 | n == 8 = "eight"
 | n == 9 = "nine"
 | otherwise = error "not a digit"

digits :: Int -> [Int]
digits n = reverse (go n)
  where go n
          | n < 10 = [n]
          | otherwise = (mod n 10) : go (div n 10)

wordNumber :: Int -> String
wordNumber n = concat . intersperse "-" . map digitToWord . digits $ n

ghci> wordNumber 531
"five-three-one"
```

## Chapter 9 - Lists

- `[]` is the type constructor for lists as well as the data constructor for the empty list in the List datatype definition:

```
data [] a = [] | a : [a]
```

- The list as a whole is a sum type because it is either an empty list or a single value consed to a list, but the second data constructor `:` is a product because it takes two arguments.
- You can pattern match on the data constructors:

```
safeTail        :: [a] -> Maybe [a]
safeTail []     = Nothing
safeTail (_:[]) = Nothing
safeTail (_:xs) = Just xs
```

- `xs` is a cute name for more than one `x` and idiomatic.
- `[1, 2, 3, 4]` is syntactic sugar for `(1 : 2 : 3 : 4 : [])`
- We can discuss lists as "cons cells" and spines. A cons cell is the list's second data constructor `a : [a]`, the result of recursively prepending a value to "more list". The spine is the chain of (:) cells linking the list together, independent of the values they hold and it can be evaluated on its own without forcing those values inside the cells. Cons cells are nested inside one another not laid out flat in a row. Remember, a cons cell is a piece of data and a pointer to the next piece of data, so it could be written `Cons 1 (Cons 2 (Cons 3 (Cons 4 Nil)))`.
- You can easily construct lists using ranges like `[1..10]`, which is equivalent to `enumFromTo 1 10`
- `take n xs` will return a list of the first `n` elements of `xs`. Taking from an empty list merely returns the empty list.
- `drop n xs` will return a list of `xs` with the first `n` elements removed.
- `splitAt n xs` will return a tuple of the first `n` elements of `xs` and the remainder of `xs`. `splitAt 3 [1..5] == ([1, 2, 3], [4, 5])`.
- `takeWhile` and `dropWhile` take a predicate and take or drop while it is `True` for the list elements. They stop at the first element that does not meet the condition. `takeWhile (<3) [1..10] == [1, 2]`.
- List comprehensions allow you to generate a new list from a list or lists.

```
[ x^2 | x <- [1..3]] == [1, 4, 9]
[ x^2 | x <- [1..5], rem x 2 == 1] == [1, 9, 25]
[ x^y | x <- [1..3], y <- [2, 3]] == [1, 1, 4, 8, 9, 27]
[ x^y | x <- [1..3], y <- [2, 3], x^y < 10] == [1, 1, 4, 8, 9]
[ (x, y) | x <- [1..3], y <- ['a', 'b']] ==
  [(1, 'a'), (1, 'b'), (2, 'a'), (2, 'b'), (3, 'a'), (3, 'b')]
```

- With multiple lists, a list comprehension generates every possible value for the first x value, then it moves to the next x value, and so on.
- Strings are list of `Char`, so you can do list comprehensions with them.

```
[x | x <- "Three Letter Acronym", elem x ['A'..'Z']] == "TLA"
acro xs = [x | x <- xs, elem x ['A'..'Z']]
vowels xs = [x | x <- xs, elem x "aeiou"]
```

- In ghci the `:sprint` command can print variables and see what is evaluated.

```
ghci> blah = enumFromTo 'a' 'z'
ghci> :sprint blah
_
ghci> take 1 blah
"a"
ghci> :sprint blah
blah = 'a' : _
-- `length` is only strict in the spine, but in ghci it evaluates everything
ghci> length blah
26
ghci> :sprint blah
blah = "abcdefghijklmnopqrstuvwxyz"
```

- "Normal form" (NF) means fully evaluated, no unevaluated parts anywhere. "Weak head normal form" (WHNF) means evaluated at least as far as the outermost data constructor (or a lambda awaiting an argument) — whatever's inside that constructor may still be unevaluated. Every normal form expression is automatically also in WHNF, but not every WHNF expression is in normal form. So WHNF is the broader category; normal form is the special case where evaluation happened to go all the way down. Example: `(1 + 1) : undefined` is in WHNF (the outer `:` constructor is reached) but not normal form (`1 + 1` and `undefined` are both left unevaluated). `\x -> x * 10` is in both WHNF and NF. `"Papu" ++ "chon"` is in neither because its outermost component is an unevaluated

function with fully applied arguments. `(1, "Papu" ++ "chon")` is in WHNF since it is fully evaluated up to its first data constructor `(,)`. When we fully define a list it is in both WHNF and NF, but when we define a list through ranges or functions it is in WHNF but not NF since the compiler only evaluates the head or first node and the cons constructor, not the value or rest of the list it contains.

```
ghci> myNum :: [Int]; myNum = [1..10]
ghci> :sprint myNum
myNum = _
ghci> take 2 myNum
[1, 2]
ghci> :sprint myNum
myNum = 1 : 2 : _
-- length evaluates the spine but not the values.
ghci> x = [1, undefined]
ghci> length x
2
-- if part of the spine is undefined, length will error
ghci> x = [1] ++ undefined ++ [3]
ghci> x
[1,*** Exception: Prelude.undefined
ghci> length x
*** Exception: Prelude.undefined
```

- In Haskell, we tend to use higher-order functions for transforming data rather than manual recursion.
- `map` or `fmap` apply a function to each element of a list and return a list of results (or Functor instead of list for `fmap`)

```
map :: (a -> b) -> [a] -> [b]
map _ []     = []
map f (x:xs) = f x : map f xs

ghci> badList = [1, 2, undefined]
ghci> map (+1) badList
[2,3,*** Exception: Prelude.undefined

ghci> take 2 $ map (+1) badList
[2,3]
```

- It doesn't need to be tail recursive since Haskell is lazy.

- `filter` takes a predicate and a list as input and it returns a list of values for which the predicate is true. `filter even [1..10] == [2,4,6,8,10]`.

```
filter :: (a -> Bool) -> [a] -> [a]
filter _ []        = []
filter pred (x:xs)
  | pred x         = x : filter pred xs
  | otherwise      = filter pred xs

-- these two are equivalent
ghci> filter (\x -> elem x "aeiou") "abracadabra"
"aaaaa"
ghci> [x | x <- "abracadabra", elem x "aeiou"]
"aaaaa"
```

- Zipping lists together allows you to combine values from multiple lists into a single list. `zip [1, 2, 3] [4, 5, 6] == [(1, 4), (2, 5), (3, 6)]`.
- Zip stops as soon as one list runs out of values and returns empty list if either list is empty.
- `unzip` will reverse this operation.
- `zipWith` takes a function and two lists and applies the function pairwise returning a list of results. `zipWith (+) [1, 2] [3, 4] = [4, 6]`.
- In type theory, a *product type* is a type made of a set of types compounded over each other. In Haskell, we represent products using tuples or data constructors with more than one argument. The "compounding" is from each type argument to the data constructor representing a value that coexists with all the other values simultaneously. Products of types represent a conjunction, "and," of those types. If you have a product of `Bool` and `Int`, your terms will *each* contain a `Bool` *and* an `Int` value.
- In type theory, a *sum type* of two types is a type whose terms are terms in either type, but not simultaneously. In Haskell, sum types are represented using the pipe, `|`, in a datatype definition. Sums of types represent a disjunction, "or" of those types. If you have a sum of `Bool` and `Int`, your terms will be *either* a `Bool` value or an `Int` value.

## Chapter 10 - Folding Lists

- Folds as a general concept are called catamorphisms. Catamorphisms are a means of deconstructing data. If the spine of a list is the structure of a list, then a fold is what can

reduce that structure. Despite the fact that a fold *can* break down this structure, the structure might be rebuilt. So a fold can return a list as a result also.

- `foldr` is "fold right" and the fold you'll most often want to use with lists.

```haskell
-- using the old type signature for lists. now it is for Foldable t
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr _ z []     = z
foldr f z (x:xs) = f x (foldr f z xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl _ z []     = z
foldl f z (x:xs) = foldl f (f z x) xs

-- Note that if `f` is `(:)` and `z` is `[]` you recreate the list
ghci> foldr f z [1,2,3]
= f 1 (foldr f z [2,3])
= f 1 (f 2 (foldr f z [3]))
= f 1 (f 2 (f 3 (foldr f z [])))
= f 1 (f 2 (f 3 z))
-- With f = (+) and z = 0, that's 1 + (2 + (3 + 0))

ghci> foldl f z [1,2,3]
= foldl f (f z 1) [2,3]
= foldl f (f (f z 1) 2) [3]
= foldl f (f (f (f z 1) 2) 3) []
= f (f (f z 1) 2) 3
-- With f = (+) and z = 0: ((0 + 1) + 2) + 3
-- You see we have to recurse to [] before producing any value at all.

-- The actual type of `foldr` now is:
foldr :: Foldable t => (a -> b -> b) -> b -> t a -> b
```

- `foldr` can work on an infinite list when given a function lazy in its second argument. `foldl`'s recursive call is a direct tail call (not wrapped as an argument to `f`), so it structurally can't short-circuit regardless of what `f` is, versus `foldr`'s call sitting inside `f x (...)`, where evaluation depends on `f`.

```haskell
myAny :: (a -> Bool) -> [a] -> Bool
myAny f xs = foldr (\x b -> f x || b) False xs

ghci> myAny even [1..]
True

ghci> myAny even (repeat 1)
-- stuck because it's processing infinite 1's

-- with a function ignoring its arguments, foldr only need be strict in
-- evaluating the list up to the spine of the first cons cell to pattern
-- match on (x:xs)
ghci> foldr (\_ _ -> 9001) 0 [1..5]
9001
ghci> xs = [1, 2, 3, undefined]
ghci> foldr (\_ _ -> 9001) 0 xs
9001
ghci> xs = [1, 2, 3] ++ undefined
ghci> foldr (\_ _ -> 9001) 0 xs
9001
ghci> xs = [undefined, undefined]
ghci> foldr (\_ _ -> 9001) 0 xs
9001
ghci> foldr (\_ _ -> 9001) 0 undefined
*** Exception: Prelude.undefined

-- const actually evaluates its first argument so is a bit more picky
ghci> foldr const 0 [1..5]
1 -- == const 1 (const 2 (const 3 (const 4 (const 5 0)))) == const 1 _
ghci> foldr const 0 [1, undefined]
1 -- but actually the second argument isn't evaluated, so this is `const 1 _`
ghci> foldr const 0 ([1,2] ++ undefined)
1 -- this is also `const 1 _`
ghci> foldr const 0 [undefined, 2]
*** Exception: Prelude.undefined

ghci> foldr (flip const) 0 [1..5]
0 -- == f_const 1 (f_const 2 (f_const 3 (f_const 4 (f_const 5 0))))
ghci> foldl (flip const) 0 [1..5]
5 -- == f_const (f_const (f_const (f_const (f_const 0 1) 2) 3) 4) 5
ghci> foldl const 0 [1..5]
0 -- == const (const (const (const (const 0 1) 2) 3) 4) 5

-- foldl must evaluate the spine. This isn't helped by a function that only
-- evaluates one argument.
```

```
ghci> xs = [1..5] ++ undefined
ghci> foldr const 0 xs
1
ghci> foldr (flip const) 0 xs
*** Exception: Prelude.undefined
ghci> foldl const 0 xs
*** Exception: Prelude.undefined
ghci> foldl (flip const) 0 xs
*** Exception: Prelude.undefined
```

- Despite the fact that `foldl` must be strict in evaluating the spine, you can give it functions that ignores its arguments and workaround undefined values. `foldl` tends to have performance problems though, and you should usually use `foldl'` which is strict.
- Consider `foldr`'s similarity with `map`. `map` applies a function to each member of a list and returns a list. a fold replaces the cons constructors with the function and reduces the list.

```
map :: (a -> b) -> [a] -> [b]
map (+1) [1,2,3] = map (+1)      1 :      2 :     3  : []
                 =          (+1) 1 : (+1) 2 : (+1 3) : []

foldr :: (a -> b -> b) -> [a] -> [b]
foldr (+) 0 [1,2,3] = foldr (+) 0 (1 :  2 :  3 : [])
                    =              1 + (2 + (3 + 0))
```

- Left folds traverse the spine in the same direction as right folds but their folding process is left associative.
- The scan functions, `scanr` and `scanl`, can show us how a fold evaluates. They are similar to folds but return a list of all the intermediate stages.

```
ghci> scanr (+) 0 [1..5]
[15,14,12,9,5,0]
ghci> scanl (+) 0 [1..5]
[0,1,3,6,10,15]
last (scanl f z xs) == foldl f z xs
head (scanr f z xs) == foldr f z xs
```

- It is hard to see the importance of associativity with arithmetic functions. Consider `foldr (^) 2 [1..3] == 1` and `foldl (^) 2 [1..3] == 64` and `foldr (:) [] [1..3] ==`

`[1,2,3]` while `fold (flip (:)) [] [1..3] == [3,2,1]`. If we didn't `flip (:)` we would get a type error because the first argument is the accumulator and thus a list while the second argument is a value.

- When we write folds, we begin by thinking about what our start value for the fold is. This is usually the identity of the function and is also the fallback value for an empty list. Next we consider the arguments. A folding function takes two arguments, `a` and `b`, where `a` is always going to be a list element and `b` is either the start value or the value accumulated.

```
-- Say we want the first three letters of each element of this list
ghci> pab = ["Pizza", "Apple", "Banana"]
ghci> f = (\a b -> take 3 a ++ b)
ghci> foldr f "" pab
"PizAppBan"
ghci> f' = (\b a -> take 3 a ++ b)
ghci> foldl f' "" pab
"BanAppPiz" -- (f' (f' (f' "" "Pizza") "Apple") "Banana")
```

- For finite lists, `foldr f z xs == foldl (flip f) z (reverse xs)`.
- You can use `foldl1` and `foldr1` when there is no proper zero value if you don't mind them crashing on an empty list.

```
foldr1 :: (a -> a -> a) -> [a] -> a
foldr1 f [x]    = x
foldr1 f (x:xs) = f x (foldr1 f xs)

foldl1 :: (a -> a -> a) -> [a] -> a
foldl1 f (x:xs) = foldl f x xs
```

- Fold is often more convenient than explicit recursion.

```haskell
-- Notice for each of these the base case is identity for the operation.
sum :: [Integer] -> Integer
sum []     = 0
sum (x:xs) = x + sum xs

length :: [a] -> Integer
length []     = 0
length (_:xs) = 1 + length xs

product :: [Integer] -> Integer
product []     = 1
product (x:xs) = x * product xs

concat :: [[a]] -> [a]
concat []     = []
concat (x:xs) = x ++ concat xs

and :: [Bool] -> Bool
and []     = True
and (x:xs) = x && and xs

or :: [Bool] -> Bool
or []     = False
or (x:xs) = x || or xs

-- now with foldr
sum :: [Integer] -> Integer
sum = foldr (+) 0

length :: [a] -> Integer
length = foldr (\_ acc -> acc +1) 0

product :: [Integer] -> Integer
product = foldr (*) 1

concat :: [[a]] -> [a]
concat = foldr (++) []

and :: [Bool] -> Bool
and = foldr (&&) True

or :: [Bool] -> Bool
or = foldr (||) False
```

- A *catamorphism* is a generalization of folds to arbitrary datatypes. Where fold allows you to break down a list into an arbitrary datatype, a catamorphism is a means of breaking down the structure of any datatype. `bool` in `Data.Bool` is a simple catamorphism for `Bool` as is `maybe` for `Maybe` and `either` for `Either`.

```
data Bool = False | True
bool :: a -> a -> Bool -> a

data Maybe a = Nothing | Just a
maybe :: b -> (a -> b) -> Maybe a -> b

data Either a b = Left a | Right b
either :: (a -> c) -> (b -> c) -> Either a b -> c
```

## Chapter 11 - Algebraic Datatypes

- A type can be thought of as an enumeration of constructors that have zero or more arguments.

```
data Bool = False | True
data [] a = []    | a : [a]
```

- `Bool` is an enumeration of two possible constructors, each of which take zero arguments (nullary constructors). The type constructor `[]` enumerates two possible constructors and one of them takes two arguments. The pipe in both data declarations denotes a *sum type*, a type that has more than one constructor inhabiting it. In addition to sum types, Haskell has *product types* which we'll cover shortly. The data constructor in a product type has more than one parameter.

- Haskell has type constructors and data constructors. Type constructors are at type level and appear in type signatures and type class declarations and instances. Types are static and resolve at compile time. Data constructors construct values at term level that you can interact with at runtime. We call them constructors, because they define a means of creating or building a type or value. Type and data constructors that take no arguments are *constants*. In the `Bool` declaration, `Bool` is a type constant, a concrete type that isn't waiting for any addition information in the form of an argument to be realized as a type. It enumerates two values that are also constants, `True` and `False`, because they take no arguments. We call `True` and `False` data constructors, but since they take no arguments

their values are already established and they are not being constructed in any meaningful sense. When a constructor takes an argument, it must be applied to become a concrete type of value. A nullary constructor is called a *type constant* to distinguish it from a type constructor that takes arguments. The list constructor must be applied to a concrete type before you have a list.

- *Kinds* are the types of types, or types one level up. They are distinguished with `*`. Something is fully-applied or concrete when its kind is `*`. When it is `* -> *`, it is still waiting to be applied. We can query the kind of a type constructor (not a data constructor) with `:kind` or `:k` in GHCI. `:k Bool` is `Bool :: *`, `:k [Int]` is `[Int] :: *`, and `:k []` is `[] :: * -> *`. Both `Bool` and `[Int]` are fully applied, so their kind signatures have no function arrows. `[]` still needs to be applied to a concrete type before it becomes a concrete type. A type like `Either a b` has kind `* -> * -> *` and is waiting to be applied to two concrete type. This is what the *constructor* of "type constructor" is referring to. We use `:type` or `:t` when finding the type of data constructors. Type constructors have kinds and data constructors have types. Constructors behave like (type or value-level) constants if they do not take an argument, and if they take arguments, they act like (type or value-level) functions that don't do anything except get applied.

- Both data constructors and type constructors begin with capital letters, but a constructor before the `=` in a datatype definition is a type constructor, while constructors after the `=` are data constructors. When data constructors take arguments, those arguments refer to other types. In Haskell we cannot choose specific values of types as type arguments. If you are a `[Bool]` list you must take values of both `True` and `False`.

- *Arity* refers to the number of arguments a function or constructor takes. A function that takes no arguments is called *nullary*. So are data constructors that take no arguments. Nullary data constructors are constant values at term level and cannot construct or represent any data other than themselves. They are values that act as witnesses of the datatype in which they are declared. Functions that take arguments might be unary or binary or take more arguments. Data constructors that take one argument are called unary. Data constructors that take more than one argument are called products. Tuples are considered the canonical product type but are called *anonymous products* because they have no name.

- Algebraic datatypes in Haskell are algebraic, because we can describe the patterns of argument structures using two basic operations: sum and product. Sum and product are most easily demonstrated in terms of *cardinality*, but it doesn't map perfectly as we can

have infinite data structures in Haskell. The cardinality of a datatype is the number of possible values it defines.

- The cardinality of a datatype roughly equates to how difficult it is to reason about.
- `Bool` has two inhabitants that are both nullary data constructors, so the cardinality of `Bool` is 2. In general, a sum datatype with nullary constructors will have a cardinality equal to the number of constructors.
- `Int` and related datatypes ( `Int8` , `Int16` , and `Int32` ) have clearly delineated upper and lower bounds. Valid `Int8` values are from -128 to 127. You can test this with `minBound :: Int8` and `maxBound :: Int8` after running `import Data.Int` , so `Int8` has cardinality 128 + 127 + 1 = 256. Anywhere you have a type of `Int8` , you have 256 possible values. This is because `Int8` is 8 bits or `2^8 == 256` values. Likewise `Int16` , `Int32` , and plain `Int` are `2^16` , `2^32` , and `2^64` values respectively.
- Unary data constructors always have the same cardinality of the type they contain. `data Goats = Goats Int deriving (Eq, Show)` has cardinality `2^64` . Anything that is a valid `Int` will be a valid argument to `Goats` . For cardinality, this means unary constructors are the identity function.
- The `newtype` keyword allows us to define a type that can only ever have a single unary data constructor. They are different from type declarations marked with the `data` keyword and from type synonym definitions marked by the `type` keyword. The cardinality of a `newtype` is the same as that of the type it contains. A `newtype` cannot be a product type, sum type, or contain nullary data constructors. It also has no runtime overhead, since it reuses the representation of the type it contains. Any difference is gone after the compiler generates the code. `newtype` declarations are similar in that any distinction between them and their underlying type is stripped away at compile time. The distinction is useful to human readers and writers of the code. For `newtype` though, you can define type class instances that differ from the instances for its underlying type. The `newtype` can rely on type instances of the type it contains for user-defined type classes if you use the `GeneralizedNewtypeDeriving` `LANGUAGE` pragma.

```haskell
-- without GeneralizedNewtypeDeriving we can call the Int instance
class TooMany a where
  tooMany :: a -> Bool

instance TooMany Int where
  tooMany n = n > 42

newtype Goats =
  Goats Int deriving (Eq, Show)

instance TooMany Goats where
  tooMany (Goats n) = tooMany n

ghci> tooMany (42 :: Int)
False
ghci> tooMany (Goats 42)
False

-- to use the PRAGMA, add this to your source file
{-# LANGUAGE GeneralizedNewtypeDeriving #-}
class TooMany a where
  tooMany :: a -> Bool

instance TooMany Int where
  tooMany n = n > 42

newtype Goats = Goats Int deriving (Eq, Show, TooMany)
```

- The `|` in sum types represents logical disjunction: "or". This is the *sum* in algebraic datatypes. To know the cardinality of a sum type, you add the cardinalities of their data constructors. Nullary constructors have a cardinality of 1.
- A product types cardinality is the product of the cardinalities of its inhabitants. A product type expresses "and". Any data constructor with two or more type arguments is a product.
- Records in Haskell are product types with additional syntax to provide convenient accessors to fields with a record.

```haskell
data Person =
  Person { name :: String
         , age :: Int }
         deriving (Eq, Show)

ghci> Person "Papu" 5
Person {name = "Papu", age = 5}
ghci> papu = Person "Papu" 5
-- The record declaration automatically declares `name` and `age` accessors
ghci> name papu
"Papu"
ghci> age papu
5
```

- All the existing algebraic rules for products and sums apply in type systems including the distributive property. Product types distribute over sum types. A type is in normal form when it is written as a sum (type) of products.

```haskell
data Fiction = Fiction deriving Show
data Nonfiction = Nonfiction deriving Show

data BookType = FictionBook Fiction
              | NonfictionBook Nonfiction
              deriving Show

type AuthorName = String

-- This is not normal form. It is not a sum of products.
data Author = Author (AuthorName, BookType)
-- We apply the distributive property to rewrite Author in normal form
data Author =
    Fiction AuthorName
  | Nonfiction AuthorName
  deriving (Eq, Show)

-- This is another common type in papers about type systems and programming
-- languages that is written in normal form. It is:
-- (Number Int) + Add (Expr Expr) + ...
data Expr =
    Number Int
  | Add Expr Expr
  | Minus Expr
  | Mult Expr Expr
  | Divide Expr Expr
```

- Try to avoid using type synonyms with unstructured data like text or binary. They are best used when you want something lighter weight than newtypes but also want your type signatures to be more explicit.
- Records are primarily syntax to create field references and don't do much heavy lifting in Haskell, but they are convenient. Whenever we have a product type that uses record accessors, we should define it separate from any sum type that is wrapping it. You want to be able to use your accessors on any value of the record type.
- The idea of a catamorphism deconstructing the datatype is generally applicable to any datatype that has values, not only lists. Below is an example of deconstructing values for a product type.

```haskell
newtype Name    = Name String deriving Show
newtype Acres   = Acres Int deriving Show

data FarmerType = DairyFarmer
                | WheatFarmer
                | SoybeanFarmer
                deriving Show

data Farmer =
  Farmer Name Acres FarmerType
  deriving Show

isDairyFarmer :: Farmer -> Bool
isDairyFarmer (Farmer _ _ DairyFarmer) = True
isDairyFarmer _ = False

-- we could do the same thing with records
data FarmerRec =
  FarmerRec { name       :: Name
            , acres      :: Acres
            , farmerType :: FarmerType }
            deriving Show

isDairyFarmerRec :: FarmerRec -> Bool
isDairyFarmerRec farmer =
  case farmerType farmer of
    DairyFarmer -> True
    _           -> False
```

- The function type is exponential. Give a function `a -> b` we can calculate the inhabitants with the formula `b ^ a`. If b and a were both `Bool`, we would have `2 ^ 2 == 4` inhabitants. `a -> b -> c` is `(c ^ b) ^ a` or just `c ^ (b * a)`.

- A kind `* -> * -> *` is known as a *higher-kinded type* and lists are an example in Haskell. Getting comfortable with higher-kinded types is important as type arguments provide a generic way to express a "hole" to be filled by consumers of your datatype later.

- When we give an operator a non-alphanumeric name it is infix by default. Alphanumeric functions are prefix by default. The same rule applies to data constructors. Any operator that starts with a colon `(:)` must be an infix type or data constructor. All infix data constructors must start with a colon. The type construct of functions `(->)` is the only infix constructor that does not start with a colon. They also cannot be `::` as this is only used in type assertions.

```haskell
data BinaryTree a =
    Leaf
  | Node (BinaryTree a) a (BinaryTree a)
  deriving (Eq, Ord, Show)

insert' :: Ord a => a -> BinaryTree a -> BinaryTree a
insert' b Leaf = Node Leaf b Leaf
insert' b (Node left a right)
  | b == a = Node left a right
  | b < a  = Node (insert' b left) a right
  | b > a  = Node left a (insert' b right)

ghci> t1 = insert' 0 Leaf
ghci> t1
Node Leaf 0 Leaf

ghci> t2 = insert' 3 t1
ghci> t2
Node Leaf 0 (Node Leaf 3 Leaf)

ghci> t3 = insert' 5 t2
ghci> t3
Node Leaf 0 (Node Leaf 3 (Node Leaf 5 Leaf))

mapTree :: (a -> b) -> BinaryTree a -> BinaryTree b
mapTree _ Leaf = Leaf
mapTree f (Node left a right) = Node (mapTree f left) (f a) (mapTree f right)

testTree' :: BinaryTree Integer
testTree' = Node (Node Leaf 3 Leaf) 1 (Node Leaf 4 Leaf)

mapExpected = Node (Node Leaf 4 Leaf) 2 (Node Leaf 5 Leaf)

mapOkay = if mapTree (+1) testTree' == mapExpected
          then print "yup OK!"
          else error "test failed!"

preorder :: BinaryTree a -> [a]
preorder Leaf = []
preorder (Node left a right) = [a] ++ preorder left ++ preorder right

inorder :: BinaryTree a -> [a]
inorder Leaf = []
inorder (Node left a right) = inorder left ++ [a] ++ inorder right
```

```haskell
postorder :: BinaryTree a -> [a]
postorder Leaf = []
postorder (Node left a right) = postorder left ++ postorder right ++ [a]

-- I solved this with a traversal
foldTree :: (a -> b -> b) -> b -> BinaryTree a -> b
foldTree _ z Leaf = z
foldTree f z tree = foldr f z (inorder tree)

-- Here is my AI companions answer that works on the tree directly
foldTree' :: (a -> b -> b) -> b -> BinaryTree a -> b
foldTree' _ z Leaf = z
foldTree' f z (Node left a right) =
 let z'  = foldTree' f z right   -- fold the right subtree first, seeded with
z
     z'' = f a z'                -- combine this node's value into that
result
  in foldTree' f z'' left         -- fold the left subtree, seeded with z''
```

- As-patterns allow you pattern match on part of something and still refer to the entire original value:

```haskell
f :: Show a => (a, b) -> IO (a, b)
f t@(a, _) = do
  print a
  return t

doubleUp :: [a] -> [a]
doubleUp [] = []
doubleUp xs@(x: _) = x : xs

-- find a subsequence only if it is in the original order
isSubseqOf :: (Eq a) => [a] -> [a] -> Bool
isSubseqOf [] xs = True
isSubseqOf xs [] = False
isSubseqOf xs'@(x:xs) (y:ys)
  | x == y = isSubseqOf xs ys
  | otherwise = isSubseqOf xs' ys

ghci> isSubseqOf "blah" "blahwoot"
True
ghci> isSubseqOf "blah" "wootblah"
True
ghci> isSubseqOf "blah" "wboloath"
True
ghci> isSubseqOf "blah" "wootbla"
False
ghci> isSubseqOf "blah" "halbwoot"
False
ghci> isSubseqOf "blah" "blawhoot"
True
```

## Chapter 12 - Signaling Adversity

- We use `Maybe` values when we don't have any sensible values to return for our intended type `a` so we want to be able to return `Nothing`.

```haskell
type Name = String
type Age = Integer

data Person = Person Name Age deriving Show

mkPerson :: Name -> Age -> Maybe Person
mkPerson name age
  | name /= "" && age >= 0 = Just $ Person name age
  | otherwise = Nothing
```

- `mkPerson` above is a *smart constructor*. It allows us to construct values of a type only when they meet certain criteria, so we know that we have a valid value, and return an explicit signal when they do not.
- One drawback of `Maybe` is that it is possible to fail but not say why you failed. To handle that we have `Either`.

```haskell
type Name = String
type Age = Integer

data Person = Person Name Age deriving Show

data PersonInvalid = NameEmpty
                   | AgeTooLow
                   deriving (Eq, Show)

mkPerson :: Name -> Age -> Either PersonInvalid Person
mkPerson name age
  | name /= "" && age >= 0 = Right $ Person name age
  | name == "" = Left NameEmpty
  | otherwise = Left AgeTooLow
```

- The convention is `Left` holds the error and `Right` is a valid result.
- But what if we need more than one error?

```
type Name = String
type Age = Integer

type ValidatePerson a = Either [PersonInvalid] a

data Person = Person Name Age deriving Show

data PersonInvalid = NameEmpty
                   | AgeTooLow
                   deriving (Eq, Show)

ageOkay :: Age -> Either [PersonInvalid] Age
ageOkay age = case age >= 0 of
  True  -> Right age
  False -> Left [AgeTooLow]

nameOkay :: Name -> Either [PersonInvalid] Name
nameOkay name = case name /= "" of
  True  -> Right name
  False -> Left [NameEmpty]

mkPerson :: Name -> Age -> ValidatePerson Person
mkPerson name age = mkPerson' (nameOkay name) (ageOkay age)

mkPerson' :: ValidatePerson Name
          -> ValidatePerson Age
          -> ValidatePerson Person
mkPerson' (Right nameOk) (Right ageOk) = Right (Person nameOk ageOk)
mkPerson' (Left badName) (Left badAge) = Left (badName ++ badAge)
mkPerson' (Left badName) _             = Left badName
mkPerson' _              (Left badAge) = Left badAge
```

- Note that you cannot hide polymorphic types from your type constructor. There is `data Unary = Unary Int` and `data Unary a = Unary a`, but for `a` to have meaning it must be introduced through the type constructor.

```haskell
import Data.Maybe

notThe :: String -> Maybe String
notThe "the" = Nothing
notThe s = Just s

replaceThe :: String -> String
replaceThe = unwords . map (fromMaybe "a" . notThe) . words

ghci> replaceThe "the cow loves us"
"a cow loves us"

lefts' :: [Either a b] -> [a]
lefts' [] = []
lefts' (x:xs) = case x of
  Left y -> y : lefts' xs
  Right y -> lefts' xs

ghci> lefts' [Left "bad", Right 10, Left "news", Left "bears", Right 5]
["bad","news","bears"]

lefts'' :: [Either a b] -> [a]
lefts'' = foldr go []
  where
    go x acc = case x of
      Left x -> x : acc
      Right x -> acc

myIterate :: (a -> a) -> a -> [a]
myIterate f x = x : myIterate f (f x)

myUnfoldr :: (b -> Maybe (a, b)) -> b -> [a]
myUnfoldr f x = case f x of
  Nothing      -> []
  Just (a, b)  -> a : myUnfoldr f b

f :: (Num b) => b -> Maybe (b, b)
f x = Just (x, x + 1)
betterIterate x = myUnfoldr f x

ghci> take 10 $ betterIterate 0
[0,1,2,3,4,5,6,7,8,9]

data BinaryTree a = Leaf | Node (BinaryTree a) a (BinaryTree a)
  deriving (Eq, Ord, Show)
```

```
unfold :: (a -> Maybe (a,b,a)) -> a -> BinaryTree b
unfold f x = case (f x) of
  Nothing  -> Leaf
  Just (a', b', a'') -> Node (unfold f a') b' (unfold f a'')

treeBuild :: Integer -> BinaryTree Integer
treeBuild n = unfold f n
 where f 0 = Nothing
       f x = Just (x-1, x-1, x-1)

ghci> treeBuild 0
Leaf
ghci> treeBuild 1
Node Leaf 0 Leaf
ghci> treeBuild 2
Node (Node Leaf 0 Leaf) 1 (Node Leaf 0 Leaf)
ghci> treeBuild 3
Node (Node (Node Leaf 0 Leaf) 1 (Node Leaf 0 Leaf)) 2 (Node (Node Leaf 0
Leaf) 1 (Node Leaf 0 Leaf))
```

- A *higher-kinded type* is any type whose kind has a function arrow in it and which can be described as a type constructor rather than as a type constant. The following types are of a higher kind than `*`:

```
Maybe  :: * -> *
[]     :: * -> *
Either :: * -> * -> *
(->)   :: * -> * -> *
```

- The following are not *higher-kinded types*:

```
Int    :: *
Char   :: *
String :: *
[Char] :: *
```

## Chapter 13 - Building Projects

- Haskell code is divided into *modules* that define and export datatypes, type synonyms, type classes, type class instances, and values defined at their top level. Haskell modules act as

namespaces. Haskell Cabal, or Common Architecture for Building Applications and Libraries, is a package manager. A *package* is a program you're building, including all its modules and dependencies, whether you've written it or you're building someone else's program. A package has *dependencies*, which are the interlinked elements of that program along with the other packages and libraries it may depend on and any tests and documentation associated with the project. Cabal exists to help organize all this and make sure all of your dependencies are properly in scope. Stack is a cross-platform program for developing Haskell projects that helps you manage both projects made up of multiple packages as well as individual packages, whereas Cabal exists primarily to describe a single package with a Cabal file ending in `.cabal`. Stack is built on top of Cabal.

- You can use `:browse` in ghci to see the functions included in a named module.
- We made a hangman game and it's in ch13 folder.

## Chapter 14 - Testing

- `Hspec` is a popular specification testing library for unit tests.

```
import Test.Hspec

main :: IO ()
main = hspec $ do
  describe "Addition" $ do
    it "1 + 1 is greater than 1" $ do
      (1 + 1) > 1 `shouldBe` True
    it "2 + 2 is equal to 4" $ do
      2 + 2 `shouldBe` 4
```

- Haskell also has `QuickCheck` which supports testing properties to assert laws or properties.

```
import Test.QuickCheck

-- ... tests above
  -- Note: asserting the type of x is necessary to generate x.
  it "x + 1 is always\
      \ greater than x" $ do
    property $ \x -> x + 1 > (x :: Int)
```

- To see what the random values for QuickCheck look like, you can use `sample` and `sample'`. The difference between the two is that `sample` prints the values while `sample'` returns them as a list of arbitrary values. You can invoke them like `sample' (arbitrary :: Gen Int)` since they take a generator of the value in question. If you don't specialize `arbitrary` to a type, it defaults to `()` and you get back only empty tuples.

- It's easy to make your own generators through combinations of `choose` and `elements` which both return a random element from a list of values.

```
genBool :: Gen Bool
genBool = choose (False, True)

genOrdering :: Gen Ordering
genOrdering = elements [LT, EQ, GT]

genCap :: Gen Char
genCap = elements ['A'..'Z']

genTuple :: (Arbitrary a, Arbitrary b) => Gen (a, b)
genTuple = do
  a <- arbitrary
  b <- arbitrary
  return (a, b)

-- Frequency returns a weighted generator. Here 1/4 Nothing, 3/4 Just a.
genMaybe :: Arbitrary a => Gen (Maybe a)
genMaybe = do
  a <- arbitrary
  frequency [ (1, return Nothing)
            , (3, return (Just a))]
```

- The above QuickCheck example uses QuickCheck with hspec, but it could be used by itself:

```
prop_additionGreater :: Int -> Bool
prop_additionGreater x = x + 1 > x

runQc :: IO ()
runQc = quickCheck prop_additionGreater
```

- So in testing our actual program, we establish a generator that only produces allowable characters and then a Property that tests the round trip.

```
allowedChars :: [Char]
allowedChars = M.keys letterToMorse

charGen :: Gen Char
charGen = elements allowedChars

prop_thereAndBackAgain :: Property
prop_thereAndBackAgain =
  forAll charGen
  (\c -> ((charToMorse c)
    >>= morseToChar) == Just c)

main :: IO ()
main = quickCheck prop_thereAndBackAgain
```

- You can load your test file to play with it in ghci with `stack ghci morse:test:morse-test` or just run `stack test` to run tests. You can also use `stack ghci --test` but it will complain if both Main.hs and Spec.hs have a `main` declared but allow you to disambiguate.
- You'll notice that to use our custom generator, we had to include `forAll charGen` in our property. You don't have to specify a generator if you declare an Arbitrary instance for a custom type. For example:

```
data Pair a b = Pair a b
  deriving (Eq, Show)

pairGen :: (Arbitrary a, Arbitrary b) => Gen (Pair a b)
pairGen = do
  a <- arbitrary
  b <- arbitrary
  return (Pair a b)

instance (Arbitrary a, Arbitrary b) => Arbitrary (Pair a b) where
  arbitrary = pairGen
```

- Remember, to sample the new generator, you need a concrete type for `a`. So you would need to `sample' (pairGen :: Gen (Pair Int String))`

- Sum types have another complication in needed `oneof` from `Test.QuickCheck.Gen` which takes a `[Gen a]`.

```
import Test.QuickCheck.Gen (oneof)

data Sum a b = First a | Second b
  deriving (Eq, Show)

-- equal odds for each
sumGenEqual :: (Arbitrary a, Arbitrary b) => Gen (Sum a b)
sumGenEqual = do
  a <- arbitrary
  b <- arbitrary
  oneof [return $ First a, return $ Second b]

-- or to save an unnecessary arbitrary generation
sumGenEqual' :: (Arbitrary a, Arbitrary b) => Gen (Sum a b)
sumGenEqual' = oneof [First <$> arbitrary, Second <$> arbitrary]

sumGenFirstPls = do
  a <- arbitrary
  b <- arbitrary
  frequency [(10, return $ First a),
             (1, return $ Second b)]

-- assign the Gen to arbitrary to make it an instance or sample it with
-- sample' (sumGenEqual :: Gen (Sum Char Int))
```

- `frequency` is exported by `Test.QuickCheck` so doesn't need a special import if we're already importing that in our test code.

## Chapter 15 - Monoid, Semigroup

- An *algebra* refers to some operations and the set they operate over. We care less about the particulars of the values or data we're working with and more about the general rules of their use. In Haskell, these algebras can be implemented by type classes - the type class defines the set of operations. When we discuss operations over a set, the set is the *type* the operations are for. The instance defines how each operations will perform for a given type or set. One of these algebras we use is *monoid*.

- A monoid is a type equipped with a binary associative operation and an identity element for that operation. For example, lists form a monoid under concatenation `(++)`, with `[]` as

the identity. For the type class Monoid, the operation is `mappend` and the identity `mempty`. Associativity means the arguments can be regrouped in different orders and give the same result. Identity means there exists some value that when we pass it as an input to our function, the operation is rendered moot and the other value is returned. `mappend x mempty = x` and `mappend mempty x = x`. `Monoid` is the type class that generalizes these laws across types.

- Type classes give us a way to recognize, organize, and use common functionalities and patterns across types that differ in some ways but also have things in common. The pattern of `Monoid` includes summation, multiplication, and list concatenation, among other things. The type class abstracts and generalizes the pattern so that you write code in terms of any type that can be monoidally combined.

```
class Semigroup m => Monoid m where
  mempty  :: m
  mappend :: m -> m -> m
  mappend = (<>)
  mconcat :: [m] -> m
  mconcat = foldr mappend mempty
```

- There is no `Monoid` for integers since there are two options: summation and multiplication. So we have the `Sum` and `Product` newtypes to wrap numeric values and signal which `Monoid` instance we want. They are built into the `Data.Monoid` module. `newtype` is a wrapper with no runtime overhead.

- When we say something *is a monoid* or can be described as *monoidal*, we mean you can define at least one law-abiding `Monoid` instance for it.

- The Abelian or commutative monoid is a variant where the binary operation is commutative. It is particularly helpful for concurrent or distributed processing.

- Laws circumscribe what constitutes a valid instance of the *algebra*, or set of operations, we're working with. We care about laws because we want our programs to be correct. Algebras are defined by their laws and are useful principally for their laws. Laws make up what algebras are. Laws provide guarantees that let us build on solid foundations when we combined programs.

- Monoids must abide by the laws of associativity and identity:

```haskell
-- left identity
mappend mempty x = x
-- right identity
mappend x mempty = x
-- associativity
mappend x (mappend y z) = mappend (mappend x y) z
-- since mappend = (<>)
x <> (y <> z) = (x <> y) <> z

mconcat = foldr mappend mempty
```

- The concept of "identity" is defined in respect to an operation. There are no identities without operations.
- `Semigroup` only has one law: associativity of `(<>)`. `Monoid` adds a second: identity, via `mempty`. Since `mappend = (<>)`, associativity carries over between the two, but the identity law is specific to `Monoid` and doesn't apply to `Semigroup` alone, since `Semigroup` has no `mempty` to state an identity law about.
- Monoids are important to folding and catamorphisms more generally. For some monoidal datatypes, the monoidal operation is less about combining the values and more about finding a summary value for the set. Appending can then be thought of as condensing any set of values to a summary value.
- Monoids are unusual among type classes in that many types have more than one. Another example is `Bool` which forms a monoid of conjunction and one of disjunction. We use the newtypes `All` and `Any` to distinguish these. `Maybe` has more than two possible monoids. Two that have an obvious relationship are `First` and `Last` which prefer the leftmost or rightmost success in a series. Both ignore `Nothing` values unless there is no `Just`. If `a` is a `Monoid`, then `Maybe a` also forms a `Monoid` under combination of values: `Nothing` acts as identity, and `Just x <> Just y = Just (x <> y)`.
- Avoid writing orphan instances at all costs. If you get an orphan instance warning from GHC, fix it. An orphan instance is when an instance is defined for a datatype and type class but not in the same module as either the declaration of the type class or the datatype. If you don't own the type class or the datatype, newtype it! Orphan instances mean your type classes start behaving differently depending on what modules are imported!
- QuickCheck is a good way to get a sense of whether or not the laws are likely to be obeyed by an instance.

```haskell
import Data.Monoid
import Test.QuickCheck

monoidAssoc :: (Eq m, Monoid m) => m -> m -> m -> Bool
monoidAssoc a b c = (a <> (b <> c)) == ((a <> b) <> c)

monoidLeftIdentity :: (Eq m, Monoid m) => m -> Bool
monoidLeftIdentity a = (mempty <> a) == a

monoidRightIdentity :: (Eq m, Monoid m) => m -> Bool
monoidRightIdentity a = (a <> mempty) == a

ghci> type S = String
ghci> type B = Bool
ghci> type MA = S -> S -> S -> B
ghci> quickCheck (monoidAssoc :: MA)
+++ OK, passed 100 tests.
ghci> quickCheck (monoidLeftIdentity :: String -> Bool)
+++ OK, passed 100 tests.
ghci> quickCheck (monoidRightIdentity :: String -> Bool)
+++ OK, passed 100 tests.
-- you could instead use `verboseCheck` to see the values being tested.
```

- A `Semigroup` is just a binary associative operation. It's basically a `Monoid` but without identity.

```haskell
class Semigroup a where
  (<>) :: a -> a -> a

semigroupAssoc :: (Eq a, Semigroup a) => a -> a -> a -> Bool
semigroupAssoc a b c = (a <> b) <> c = a <> (b <> c)
```

- `NonEmpty` which specifies a non-empty List type is a good example of a `Semigroup` instance without a `Monoid` since it lacks an identity. You need to `import Data.List.NonEmpty` for `NonEmpty` and `import Data.Semigroup` for `Semigroup`.
- When we talk about the *strength* of an algebra, we usually mean the number of operations it provides. `Semigroup` is weaker than `Monoid`. But sometimes you need a `NonEmpty` List more than you need `mempty`. A *magma* is weaker than a `Semigroup` and removes the associativity requirement.
- Definitions to review:

- A *monoid* is a set that is closed under an associative binary operation and has an identity element. *Closed* means `mappend :: m -> m -> m` such that your arguments and output will always inhabit the same type (set).
- A *semigroup* is a set that is closed under an associative binary operation, and nothing else.
- Laws are rules about how an algebra or structure should behave. These are needed in part to make abstraction over the commonalities of different instantiations of the same sort of algebra possible and practical. This is critical to have abstractions that aren't unpleasantly surprising.
- An *algebra* is variously:
    1. School algebra, such as that taught in primary and secondary school. This usually entails the balancing of polynomial equations and learning how functions and graphs work.
    2. The study of number systems and operations within them. This will typically entail a particular area such as groups or rings. This is what mathematicians commonly mean by "algebra." This is sometimes disambiguated by being referred to as abstract algebra.
    3. A third and final way algebra is used is to refer to a vector space over a field with a multiplication operation.
    - When Haskellers refer to algebras, they're usually talking about a somewhat informal notion of operations over a type and its laws, such as with semigroups, monoids, groups, semirings, and rings.

```haskell
-- src/Monoid.hs
import Data.Semigroup
import Data.Monoid
import Test.QuickCheck

-- property tests for the laws
semigroupAssoc :: (Eq m, Semigroup m) => m -> m -> m -> Bool
semigroupAssoc a b c = (a <> (b <> c)) == ((a <> b) <> c)

monoidLeftIdentity :: (Eq m, Monoid m) => m -> Bool
monoidLeftIdentity a = (mempty <> a) == a

monoidRightIdentity :: (Eq m, Monoid m) => m -> Bool
monoidRightIdentity a = (a <> mempty) == a

-- short names
sa :: (Eq m, Semigroup m) => m -> m -> m -> Bool
sa = semigroupAssoc
mli :: (Eq m, Monoid m) => m -> Bool
mli = monoidLeftIdentity
mlr :: (Eq m, Monoid m) => m -> Bool
mlr = monoidRightIdentity

-- Two a b
data Two a b = Two a b deriving (Eq, Show)

instance (Semigroup a, Semigroup b) => Semigroup (Two a b) where
  Two a b <> Two a' b' = Two (a <> a') (b <> b')

instance (Monoid a, Monoid b) => Monoid (Two a b) where
  mempty = Two mempty mempty
  mappend = (<>)

instance (Arbitrary a, Arbitrary b) => Arbitrary (Two a b) where
  arbitrary = do
    a <- arbitrary
    b <- arbitrary
    return (Two a b)

type TwoAssoc a b = Two a b -> Two a b -> Two a b -> Bool

type TwoIdent a b = Two a b -> Bool

testTwoMonoid :: IO ()
testTwoMonoid = do
```

```
  quickCheck (sa :: TwoAssoc String String)
  quickCheck (mli :: TwoIdent String String)
  quickCheck (mlr :: TwoIdent String String)
```

## Chapter 16 - Functors

- A functor is a way to apply a function over or around some structure that we do not want to alter. We want to apply the function to the value that is "inside" some structure and leave the structure alone. For example, when you map over a list you transform elements of the list without changing the list structure. The length of the list after mapping a function over it will always be the same. No elements are removed or added, only transformed. The type class `Functor` generalizes this pattern so that we can use the basic idea with many structures and not just lists.

```
class Functor f where
  fmap :: (a -> b) -> f a -> f b

-- fmap works on a variety of types that are functors
fmap ::  (a -> b) ->          f a ->          f b
     ::  (a -> b) ->        [ ] a ->        [ ] b
     ::  (a -> b) ->      Maybe a ->      Maybe b
     ::  (a -> b) ->   Either e a ->   Either e b
     ::  (a -> b) ->       (e,) a ->       (e,) b
     ::  (a -> b) ->   Identity a ->   Identity b
     ::  (a -> b) -> Constant e a -> Constant e b
```

- `f` in the `Functor` type must have kind `* -> *`. A type with kind `* -> *` is awaiting application to one more type of kind `*` before it becomes concrete. In `fmap`'s signature, `f` is applied to `a` and to `b`, and both `f a` and `f b` must have kind `*` (they're the actual argument and result types), so `f` itself must have kind `* -> *`, one argument away from being concrete.
- Recall that we can refer to a fully applied type of kind `*` as a type constant.
- Remember that the name of the variable before the `where` in a type class definition binds the occurrences of that name throughout the definition, so you cannot introduce a new variable after the `where` that isn't before it. Note that this applies to the `f` in `Functor` definition, but `a` and `b` are fresh method local variables. `f` also consistently takes one argument through the definition.

- The infix version of `fmap` is `(<$>)`.
- `Functor` is a type class for function application "over" or "through" some structure `f` that we want to ignore and leave untouched. Notice the similarity between `(<$>)` and `($)`:

```
(<$>) :: Functor f => (a -> b) -> f a -> f b
($)   ::              (a -> b) ->   a ->   b
```

- Notice that if we wanted to declare an instance for type `FixMePls a` we would start `instance Functor FixMePls` with no reference to a. `Functor` expects the unapplied type of kind `* -> *`. `FixMePls a` is kind `*`.
- Instances of the `Functor` type class should abide by two basic laws. Identity: `fmap id = id`. Composition: `fmap (f . g) = fmap f . fmap g`.
- Both laws touch on the fact that `fmap` should be structure preserving. The `f` is untouched by `fmap`. `Functor` is a way of lifting over structure (mapping) in such a manner that you don't have to care about the structure, because you're not allowed to touch the structure anyway. If you need to change the structure, use a plain old function. The point of `Functor` is to reify and be able to talk about cases where we want to reuse functions in the presence of more structure and ignore that structure.
- `(fmap . fmap) f` will apply f to an inner functor, for example within the `Just` of a List of `Maybe` values. And `(fmap . fmap . fmap) f` will apply f to the `String` within a `Maybe` within a `List`, for example.
- So `(,)` (the Tuple constructor) and `Either` are both kind `* -> * -> *` but we have `Functor` instances for them despite the fact that `Functor` takes kind `* -> *`. The trick is partial application. `Either String` is kind `* -> *`. Basically, the first type argument is set and ignored by the transformation; it's considered part of the structure that remains unchanged. `fmap` across `Either` only modifies a `Right` and ignores `Left`.
- We know the `Functor` laws are `fmap id = id` and `fmap (p . q) = (fmap p) . (fmap q)`, so let's test those properties with QuickCheck:

```
functorIdentity :: (Functor f, Eq (f a)) => f a -> Bool
functorIdentity f = fmap id f == f

functorCompose :: (Eq (f c), Functor f) =>
                  (a -> b) -> (b -> c) -> f a -> Bool
functorCompose f g x =
  (fmap g (fmap f x)) == (fmap (g . f) x)

-- As long as we provide these concrete instances, we can test them.
ghci> :{
let f :: [Int] -> Bool
    f x = functorIdentity x
:}
ghci> quickCheck f
+++ OK, passed 100 tests

ghci> c = functorCompose (+1) (*2)
ghci> li x = c (x :: [Int])
ghci> quickCheck li
+++ OK, passed 100 tests
```

- `nat` or *natural transformations* are similar to `fmap` but they transform the structure instead of the value. Like `nat :: (f -> g) -> f a -> g a`. But that won't work, because higher-kinded types can't be an argument to the function type. But if we enable `{-# LANGUAGE RankNTypes #-}` we can declare `type Nat f g = forall a . f a -> g a`.

```
maybeToList :: Nat Maybe []
maybeToList Nothing = []
maybeToList (Just a) = [a]
```

- Definitions

  1. *Higher-kinded polymorphism* is polymorphism that has a type variable abstracting over types of a higher kind. `Functor` is an example of higher- kinded polymorphism, because the kind of the `f` parameter is `* -> *`. Another example of higher-kinded polymorphism would be a datatype having a parameter to a type constructor that is of a higher kind, such as: `data Weird f a = Weird (f a)` where the kinds of the types involved are `a :: *`, `f :: * -> *`, `Weird :: (* -> *) -> * -> *`. Here both `Weird` and `f` are higher-kinded, with `Weird` being an example of higher-kinded polymorphism.

2. *Functor* is a mapping between categories. In Haskell, this manifests as a type class that generalizes the concept of `map`: it takes a function `(a -> b)` and lifts it into a different type. This conventionally implies some notion of a function that can be applied to a value with more structure than the unlifted function was originally designed for. The additional structure is represented by the use of a higher-kinded type `f` introduced by the definition of the `Functor` type class. One should be careful not to confuse this intuition for it necessarily being exclusively about containers or data structures. There's a `Functor` of functions, and many exotic types have a lawful `Functor` instance, as well.

3. There are a couple of way people commonly think about *lifting*. One is that we can lift a function into a context. Another is that we lift a function over some layer of structure to apply it. The effect is the same. `fmap (+1) $ Just 1` lifts the function into the `Maybe` context in order to apply it, and `fmap (+1) [1, 2, 3]` lifts into the list context. It can be helpful to think of it in terms of lifting the function into a context, because it's the context we've lifted the function into that determines how it will get applied (to one value or, recursively, to many, for example). The context is the datatype, the definition of the datatype, and the `Functor` instance we have for that datatype. It's also the contexts that determine what happens when we try to apply a function to an `a` that isn't there.

   But we also speak more casually about lifting over, as in `fmap` lifts a function over a data constructor. This works if you think of the data constructor as a layer of structure. The function hops over that layer and applies to what's inside, if anything.

   More precisely, lifting means applying a type constructor to a type, as in taking an `a` type variable and apply an `f` type constructor to it to get an `f a`. Keeping this definition in mind will be helpful. Remember to *follow the types* rather than getting too caught up in the web of metaphor.

## Chapter 17 - Applicative

- The `Applicative` type class allows for function application lifted over structure like `Functor`, but the function we're applying is also embedded in some structure.

```haskell
class Functor f => Applicative f where
  pure :: a -> f a
  (<*>) :: f (a -> b) -> f a -> f b

-- note the following
fmap f x = pure f <*> x

-- consider the following
($)   ::   (a -> b) ->   a ->   b
(<$>) ::   (a -> b) -> f a -> f b
(<*>) :: f (a -> b) -> f a -> f b

-- additional functions in `Control.Applicative`
liftA :: Applicative f => (a -> b) -> f a -> f b
liftA2 :: Applicative f => (a -> b -> c) -> f a -> f b -> f c
liftA3 :: Applicative f => (a -> b -> c -> d) -> f a -> f b -> f c -> f d
```

- `(<*>)` is called "apply" or just "ap." Note how much `<*>` looks like `fmap` in type signature, except the function is within a structure.
- `Applicative` acts like a combination of Monoid and Functor. When you `<*>` across a Tuple, the first two values are joined with `mappend` and the second value gets the function applied to it: `("Woo", (+1)) <*> (" Hoo!", 0)` is `("Woo Hoo!", 1)`.

```haskell
instance (Monoid a, Monoid b) => Monoid (a, b) where
  mempty = (mempty, mempty)
  (a, b) `mappend` (a', b') = (a `mappend` a', b `mappend` b')

instance Monoid a => Applicative ((,) a) where
  pure x = (mempty, x)
  (u, f) <*> (v, x) = (u `mappend` v, f x)
```

- With the List `Applicative` we can map a plurality of functions over a plurality of values: `[(+1), (*2)] <*> [2, 4] == [3,5,4,8]`.

```haskell
ghci> (,) <$> [1, 2] <*> [3, 4] == [(1,3),(1,4),(2,3),(2,4)]
ghci> liftA2 (,) [1, 2] [3, 4] == [(1,3),(1,4),(2,3),(2,4)]
ghci> (+) <$> [1, 2] <*> [3, 4] == [4,6,5,7]
ghci> liftA2 (+) [1, 2] [3, 4] == [4,6,5,7]
```

- Applicative Laws

1. Identity

```
pure id <*> v = v

pure id <*> [1..5] == [1..5]
```

2. Composition - the result of composing our function first and then applying them and the result of applying the functions first and then composing them should be the same.

```
pure (.) <*> u <*> v <*> w = u <*> (v <*> w)

-- This one looks weird without an example. These two are equal.
pure (.) <*> [(+1)] <*> [(*2)] <*> [1, 2, 3] == [3, 5, 7]
[(+1)] <*> ([(*2)] <*> [1, 2, 3]) == [3, 5, 7]
```

3. Homomorphism - the effect of applying a function that is embedded in some structure to a value that is embedded in some structure should be the same as applying a function to a value without affecting any outside structure. A *homomorphism* is a structure-preserving map between two algebraic structures. The general idea is that applying the function doesn't change the structure around the values.

```
pure f <*> pure x = pure (f x)

pure (+1) <*> pure 1 == pure ((+1) 1) == pure 2
```

4. Interchange

```
u <*> pure y = pure ($ y) <*> u

-- recall ($ 2) == \f -> f $ 2
Just (+2) <*> pure 2 == 4 == pure ($ 2) <*> Just (+2)
```

- We will use checkers to test laws. You give it a tuple of three value types embedded in a structural type to test Applicative and Monad laws. It doesn't use the value you pass it other than as a witness.
- Some basic exercises that follow the Applicative laws:

```haskell
import qualified Test.QuickCheck as QC
import Test.QuickCheck.Checkers
import Test.QuickCheck.Classes

data List a = Nil | Cons a (List a)
  deriving (Eq, Show)

instance Functor List where
  fmap _ Nil = Nil
  fmap f (Cons x xs) = Cons (f x) (fmap f xs)

instance Applicative List where
  pure a = Cons a (Nil)
  Nil <*> _ = Nil
  _ <*> Nil = Nil
  Cons f xs <*> Cons y ys = fmap f (Cons y ys) `append` (xs <*> (Cons y ys))

instance QC.Arbitrary a => QC.Arbitrary (List a) where
  arbitrary = QC.oneof [return Nil, Cons <$> QC.arbitrary <*> QC.arbitrary]

instance Eq a => EqProp (List a) where
  (=-=) = eq

append :: List a -> List a -> List a
append Nil ys = ys
append (Cons x xs) ys = Cons x $ xs `append` ys

-- There is also an implementation of ZipList' in src/Applicative.hs

-- Validation is like Either but monoidally combines its errors
data Validation e a = Failure e | Success a
  deriving (Eq, Show)

instance Functor (Validation e) where
  fmap _ (Failure f) = Failure f
  fmap f (Success a) = Success $ f a

instance Monoid e => Applicative (Validation e) where
  pure x = Success x
  (Failure f) <*> (Failure f') =  Failure $ f <> f'
  (Failure f) <*> _ = Failure f
  _ <*> (Failure f) = Failure f
  Success f <*> Success x = Success (f x)

instance (QC.Arbitrary a, QC.Arbitrary e) =>
```

```
          QC.Arbitrary (Validation e a) where
  arbitrary = QC.oneof [Failure <$> QC.arbitrary
                       ,Success <$> QC.arbitrary]

instance (Eq e , Eq a) => EqProp (Validation e a) where
  (=-=) = eq

runQC :: IO ()
runQC = do
  quickBatch (applicative (Cons (1, 2, 3) Nil :: List (Int, Int, Int)))
  quickBatch (applicative
    (Success (1, 2, 3) :: Validation [String] (Int, Int, Int)))
```

- Definitions

  1. `Applicative` can be thought of as characterizing monoidal functors in Haskell. For a Haskeller's purposes, it's a way to functorially apply a function that is embedded in a structure `f` of the same type as the value you're mapping it over:

     ```
     fmap  ::   (a -> b) -> f a -> f b
     (<*>) :: f (a -> b) -> f a -> f b
     ```

## Chapter 18 - Monads

- A monad is an applicative functor with some unique features that make it more powerful than either alone.

```haskell
class Applicative m => Monad m where
  (>>=) :: m a -> (a -> m b) -> m b
  (>>) :: m a -> m b -> m b
  return :: a -> m a

-- fmap can be written using monadic functions
fmap f xs = xs >>= return . f

-- monad as has `join`, which is like `concat` for various structure
import Control.Monad (join)
join :: Monad m => m (m a) -> m a

-- you can write bind in terms of fmap and join. Here is bind flipped:
bind :: Monad m => (a -> m b) -> m a -> m b
bind f ma = join (fmap f ma)
-- or with ($)
bind f ma = join $ fmap f ma
-- or point free
bind f = join . fmap f
```

- `return` is just `pure` with another name. `(>>)` is sometimes called the sequencing operator but doesn't have an official name. The sequencing operator sequences two actions while discarding any resulting value from the first. And `>>=` is pronounced bind. Bind makes monad special and is the minimal implementation of the type class.
- Monad also has its own lift functions: `liftM`, `liftM2` and `liftM3`. They are the same as `liftA`, `liftA2`, and `liftA3` but predate the Applicative.
- You may remember `zipWith`. The type signature of `liftA2` or `liftM2` looks the same with `zipWith` specialized to Lists. There is also a `zipWith3` that is like `liftA3` / `liftM3`. There is a big difference though. `zipWith` combines elements pairwise like `ZipList` where lifting combines values into the Cartesian product.

```haskell
ghci> :t zipWith
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
ghci> zipWith (+) [3, 4] [5, 6]
[8, 10]
ghci> liftA2 (+) [3, 4] [5, 6]
[8,9,9,10]
```

- We didn't mention it earlier, but Applicative also has its own sequencing operator `(*>)` which is like `(>>)`.
- `do` syntax works with any monad but is commonly seen with `IO`. `do` syntax desugars as follows:

```haskell
sequencing :: IO ()
sequencing = do
  putStrLn "blah"
  putStrLn "another thing"

sequencing' :: IO ()
sequencing' =
  putStrLn "blah" >>
  putStrLn "another thing"

binding :: IO ()
binding = do
  name <- getLine
  putStrLn name

binding :: IO ()
binding =
  getLine >>= putStrLn

bindingAndSequencing :: IO ()
bindingAndSequencing = do
  putStrLn "name pls:"
  name <- getLine
  putStrLn ("y hello thar: " ++ name)

bindingAndSequencing' :: IO ()
bindingAndSequencing' =
  putStrLn "name pls:" >>
  getLine >>=
  \name ->
    putStrLn ("y hello thar: " ++ name)

twoBinds :: IO ()
twoBinds = do
  putStrLn "name pls:"
  name <- getLine
  putStrLn "age pls:"
  age <- getLine
  putStrLn ("y hello thar: "
            ++ name ++ " who is: "
            ++ age ++ " years old.")

twoBinds' :: IO ()
twoBinds' =
  putStrLn "name pls:" >>
```

```
  getLine >>=
    \name ->
      putStrLn "age pls:" >>
      getLine >>=
        \age ->
          putStrLn ("y hello thar: "
                    ++ name ++ " who is: "
                    ++ age ++ " years old.")
```

- Monad examples:

```
twiceWhenEven :: [Integer] -> [Integer]
twiceWhenEven xs = do
  x <- xs   -- process each element of xs
  if even x
    then [x*x, x*x]
    else []
```

- With the `Maybe Applicative`, each `Maybe` computation fails or succeeds idependent of one another. You're lifting functions that are also `Just` or `Nothing` over `Maybe` values.
- With the `Maybe Monad`, computations contributing to the final result can choose to return `Nothing` based on previous computations.
- Monad Laws

```haskell
-- right identity
m >>= return   = m
-- left identity
return x >>= f = f x
-- Basically `return` should be neutral and not perform any computation.

-- Associativity
(m >>= f) >>= g == m >>= (\x -> f x >>= g)

-- Composition is special because of the types being a -> m b.
-- Here we fmap f inside the structure to produce an `m (m c)` then join.
mcomp :: Monad m => (b -> m c) -> (a -> m b) -> a -> m c
mcomp f g a = join (f <$> (g a))
-- but we could just bind instead of joining the results of fmap.
mcomp' :: Monad m => (b -> m c) -> (a -> m b) -> a -> m c
mcomp' f g a = g a >>= f
-- this is even easier to write with `>=>` or Kleisli composition.
-- it's simply function composition written in terms of bind.
-- but we have to flip the arguments around.
import Control.Monad
(>=>) :: Monad m => (a -> m b) -> (b -> m c) -> a -> m c
flip (.) ::         (a ->   b) -> (b ->   c) -> a ->   c
-- Kleisli composition applies functions left to right:
mcomp'' f g a = (g >=> f) a
```

- Let's test these for our own version of `Either`.

```haskell
import Test.QuickCheck
import Test.QuickCheck.Checkers
import Test.QuickCheck.Classes

data Sum a b = First a | Second b
  deriving (Eq, Show)

instance Functor (Sum a) where
  fmap _ (First l) = First l
  fmap f (Second r) = Second $ f r

instance Applicative (Sum a) where
  pure = Second
  First l <*> _ = First l
  _ <*> First l = First l
  Second f <*> Second x = Second $ f x

instance Monad (Sum a) where
  return = pure
  First l >>= _ = First l
  Second r >>= k = k r

instance (Arbitrary e, Arbitrary a) => Arbitrary (Sum e a) where
  arbitrary = frequency [(1, First <$> arbitrary)
                        ,(3, Second <$> arbitrary)]

instance (Eq e, Eq a) => EqProp (Sum e a) where
  (=-=) = eq

runQC :: IO ()
runQC = do
  let trigger :: Sum String (Int, Int, Int)
      trigger = undefined
  quickBatch $ functor trigger
  quickBatch $ applicative trigger
  quickBatch $ monad trigger
```

- Definitions

  1. *Monad* is a type class reifying an abstractoin that is commonly used in Haskell. Instead of an oridinary function of type `a` to `b`, you're functorially applying a function that produces more structure itself and using `join` to reduce the nested structure that results.

```
fmap  ::   (a -> b)   -> f a -> f b
(<*>) :: f (a -> b)   -> f a -> f b
(=<<) ::   (a -> f b) -> f a -> f b
```

2. A *monadic function* is one that generates more structure after having already been lifted over monadic structure. Contrast the function arguments to `fmap` and `>>=`:

```
fmap :: (a -> b) -> f a -> f b
(>>=) :: m a -> (a -> m b) -> m b
```

The significant difference is that the result of `>>=` is `m b` and requires `join`ing the result after lifting the function over `m`. What does this mean? That depends on the `Monad` instance.

The distinction can be seen with oridinary function composition and Kleisli composition, as well:

```
(.)   ::            (b ->   c) -> (a ->   b) -> a ->   c
(>=>) :: Monad m => (a -> m b) -> (b -> m c) -> a -> m c
```

## Chapter 19 - Applying Structure

## Chapter 20 - Foldable

- A list fold is a way to reduce the values inside a list to one summary value by recursively applying some function. The folding function is always dependent on some `Monoid` instance. Generalizing catamorphisms to other datatypes depends on understanding the monoids of those structures and, in some cases, making them explicit.

```haskell
class Foldable (t :: * -> *) where
  fold :: Monoid m => t m -> m
  foldMap :: Monoid m => (a -> m) -> t a -> m
  {-# MINIMAL foldMap | foldr #-}

data Identity a = Identity a
instance Foldable Identity where
  foldr f z (Identity x) = f x z
  foldl f z (Identity x) = f z x
  foldMap f (Identity x) = f x

-- Note Maybe doesn't have a default Monoid, so you need to specify a
-- monoidal type to use `foldMap` like `foldMap (+1) Nada :: Sum Int`(==0).
data Optional a = Nada | Yep a
instance Foldable Optional where
  foldr _ z Nada = z
  foldr f z (Yep x) = f x z
  foldl _ z Nada = z
  foldl f z (Yep x) = f z x
  foldMap _ Nada = mempty
  foldMap f (Yep x) = f x

-- `Foldable` type class has other useful functions.
-- | List of elements of a structure, from left to right.
toList :: Foldable t => t a -> [a]
-- | Test whether the structure is empty.
null :: Foldable t => t a -> Bool
-- | Returns the size/length of a finite structure as an 'Int`.
-- It's an important quirk for tuples that these only use the right element.
length :: Foldable t => t a -> Int
-- | Does the element occur in the structure?
-- Note also that this only compares the right value of tuples and the Right
-- value of an Either. Left tuple values and Left Either's are False.
elem :: (Eq a, Foldable t) => a -> t a -> Bool
-- | The largest element of a non-empty structure.
maximum :: (Foldable t, Ord a) => t a -> a
-- | The least element of a non-empty structure.
minimum :: (Foldable t, Ord a) => t a -> a
-- | sum and product do what you would expect given their names.
sum :: (Foldable t, Num a) => t a -> a
product :: (Foldable t, Num a) => t a -> a
```

# Chapter 21 - Traversable

- `Traversable` allows you to transform elements inside a structure like a functor, producing applicative effects along the way, and lift those potentially multiple instances of applicative structure outside of the traversable structure. It is commonly described as a way to traverse a data structure, mapping a function inside a structure while accumulating applicative contexts in the process.

```haskell
class (Functor t, Foldable t) => Traversable t where
  traverse :: Applicative f => (a -> f b) -> t a -> f (t b)
  traverse f = sequenceA . fmap f
  -- | Evaluate each action in the structure from left to right and
  -- collect the results.
  sequenceA :: Applicative f => t (f a) -> f (t a)
  sequenceA = traverse id
  {-# MINIMAL traverse | sequenceA #-}
```

- The `traverse` function maps each element of a structure to an action, evaluates the actions from left to right, and collects the results. If you find yourself with a type like `[IO a]`, it's possible you made a mistake and used `fmap` where you needed `traverse`. As you get comfortable with Haskell, you'll learn to recognize that if you use `fmap` to get type `t (f b)`, when what you want is `f (t b)`, you need to use `Traversable`.

- `sequenceA` allows you to flip two contexts or structures. It doesn't by itself allow you to apply any function to the `a` value inside the structure. `sequenceA` can turn a List of Maybe values into a Maybe List. If any Maybe is Nothing, then the final value is Nothing. Compare to `catMaybes` from `Data.Maybe` that turns a List of Maybe values into a List of values and ignores Nothing. `sequence` is the old form of `sequenceA` but was specialized to monads and Lists.

- Anytime you are combining `sequenceA . fmap f`, you could instead use `traverse`. `traverse Just [1, 2, 3] == Just [1, 2, 3]`. Also `mapM` is just `traverse` and you may see it in older code. It is specialized to monads and Lists.

- We can use `Traversable` to express `fmap` and `foldMap`. `fmap` is just `runIdentity $ traverse (Identity . f)` and `foldMap` is just `getConstant $ traverse (Constant . f)` across a monoidal traversable.

```haskell
data Either' a b = Left' a | Right' b
  deriving (Eq, Ord, Show)

instance Functor (Either' a) where
  fmap _ (Left' x) = Left' x
  fmap f (Right' y) = Right' (f y)

instance Applicative (Either' e) where
  pure           = Right'
  Left'  e <*> _ = Left' e
  Right' f <*> r = fmap f r

instance Foldable (Either' a) where
  foldMap _ (Left' _) = mempty
  foldMap f (Right' y) = f y

  foldr _ z (Left' _) = z
  foldr f z (Right' y) = f y z

instance Traversable (Either' a) where
  traverse _ (Left' x) = pure (Left' x)
  traverse f (Right' y) = Right' <$> f y
```

- Traversable Laws

  1. Naturality: `t . traverse f = traverse (t . f)`.

     This law tells us that function composition behaves in unsurprising ways with respect to a traversed function. Since a traversed function `f` generates the structure that appears on the "outside" of the `traverse` operation, there's no reason we can't float a function over the structure into the traversal itself.

  2. Identity: `traverse Identity = Identity`.

     This law states that traversing the data constructor of the `Identity` type over a value will produce the same result as just putting the value in `Identity`. This tells us that `Identity` represents a structural identity for traversing data. This is another way of saying that a `Traversable` instance cannot add or inject any structure or effects.

  3. Composition:

```
traverse (Compose . fmap g . f) = Compose . fmap (traverse g) . traverse
f
```

This law demonstrates how we can collapse sequential traversals into a single traversal by taking advantage of the `Compose` datatype which combines structure.

- `sequenceA` Laws
  1. Naturality: `t . sequenceA = sequenceA . fmap t`.
  2. Identity: `sequenceA . fmap Identity = Identity`.
  3. Composition:

```
sequenceA . fmap Compose = Compose . fmap sequenceA . sequenceA
```

- In the exercises we defined all the instances for Tree among other things, and it was instructive but a lot easier than the book promised:

```haskell
data Tree a = Empty | Leaf a | Node (Tree a) a (Tree a)
  deriving (Eq, Show)

instance Functor Tree where
  fmap _ Empty          = Empty
  fmap f (Leaf a)       = Leaf $ f a
  fmap f (Node t1 a t2) = Node (fmap f t1) (f a) (fmap f t2)

instance Foldable Tree where
  foldMap _ Empty          = mempty
  foldMap f (Leaf a)       = f a
  foldMap f (Node t1 a t2) = (foldMap f t1) <> (f a) <> (foldMap f t2)
  foldr _ z Empty          = z
  foldr f z (Leaf a)       = f a z
  foldr f z (Node t1 a t2) = foldr f (f a (foldr f z t2)) t1

instance Traversable Tree where
  traverse _ Empty          = pure Empty
  traverse f (Leaf a)       = Leaf <$> f a
  traverse f (Node t1 a t2) = Node <$> traverse f t1 <*> f a <*> traverse f
t2

-- We had to skew this towards short trees for the tests to complete on a
-- reasonable time scale.
instance Arbitrary a => Arbitrary (Tree a) where
  arbitrary = frequency
    [
      (3, return Empty)
    , (3, Leaf <$> arbitrary)
    , (1, Node <$> arbitrary <*> arbitrary <*> arbitrary)
    ]

instance Eq a => EqProp (Tree a) where
  (=-=) = eq

runQC :: IO ()
runQC = do
  let trigger :: Tree (Sum Int, Sum Int, Int, Sum Int)
      trigger = undefined
  quickBatch (traversable trigger)
```

## Chapter 22 - Reader

- The Functor of functions is composition. The Applicative and Monad both feed the *same* input to every function involved (not sequentially, chaining one's output into the next, but in parallel, each function independently receiving the shared argument), then combine the results. `((+) <$> (*2) <*> (+10)) 3` feeds `3` into both `(*2)` and `(+10)` independently, then adds the two results, matching a `do` block that binds `a <- (*2)`, `b <- (+10)`, then returns `a + b`.
- The basic idea of `Reader` is that it is a way of stringing functions together when all those functions are awaiting one input from a shared environment. We use this most often when we have a constant value that we will obtain from somewhere outside our program that will be an argument to a whole bunch of functions. Using `Reader` allows us to not pass that argument around explicitly. Normally `Reader` refers to the `Monad` instance of functions.

```haskell
newtype Reader r a =
  Reader { runReader :: r -> a }

-- it's just composing (f . ra)
instance Functor (Reader r) where
  fmap :: (a -> b) -> Reader r a -> Reader r b
  fmap f (Reader ra) = Reader $ \r -> f (ra r)

{-# LANGUAGE InstanceSigs #-}
instance Applicative (Reader r) where
  pure ::  a -> Reader r a
  pure a = Reader $ \_ -> a

  (<*>) :: Reader r (a -> b) -> Reader r a -> Reader r b
  (Reader rab) <*> (Reader ra) = Reader $ \r -> rab r (ra r)

instance Monad (Reader r) where
  return = pure

  (>>=) :: Reader r a -> (a -> Reader r b) -> Reader r b
  (Reader ra) >>= aRb = Reader $ \r -> runReader (aRb (ra r)) r
```

- In practice, you tend to see `ReaderT` and not `Reader`. Also, if you have a `Reader`, it's of a record of several (possibly many) values that you are getting out of the `Reader`.
- Definitions

1. A *monad transformers* is a special type that takes a monad as an argument and returns a monad as a result. It allows us to combine two monads into one that share the behaviors of both, such as allowing us to add exception handling to the `State` monad. It is somewhat common to build a *stack* of transformers that creates one large monad with features from several different monads. For example, we can roll `Reader`, `Either`, and `IO` together to get a monad that captures the behavior of waiting for an argument that will get passed around to multiple functions but is likely to come in via some kind of IO action and has a possibility of failure that we might like to catch. Often, this stack will be given a type alias for convenience.

## Chapter 23 - State

- We can think of state as data that exists in addition to the inputs and outputs of our functions; data that can potentially change after each function is evaluated. If you need in-place mutation, then the `ST` type is what you want. `State` is appropriate when you want to express your program in terms of values that potentially vary with each evaluation step, which can be read and modified, but don't otherwise have specific operational constraints. `State` in Haskell:

    1. Doesn't require `IO`.
    2. Is limited only to the data in our `State` container.
    3. Maintains referential transparency.
    4. Is explicit in the types of our functions.
- `State` is often used for things like random number generators, solvers, games, and carrying working memory while traversing a data structure. The polymorphism means you don't have to make a new state for each possible instantiation of `s` and `a`.

```haskell
newtype State s a = State { runState :: s -> (a, s) }
State :: (s -> (a, s)) -> State s a
runState :: State s a -> s -> (a, s)
-- notice `random` looks a lot like `State`.
random :: (Random a) => StdGen -> (a, StdGen)
```

- We will implement `State` under the name `Moi` to show its instances:

```haskell
{-# LANGUAGE InstanceSigs #-}

module StateExample where

-- Moi is State
newtype Moi s a = Moi { runMoi :: s -> (a, s) }

instance Functor (Moi s) where
  fmap :: (a -> b) -> Moi s a -> Moi s b
  fmap f (Moi g) = Moi $ \s -> let (a, s') = g s
                               in (f a, s')

instance Applicative (Moi s) where
  pure :: a -> Moi s a
  pure a = Moi $ \s -> (a, s)

  (<*>) :: Moi s (a -> b) -> Moi s a -> Moi s b
  (Moi f) <*> (Moi g) = Moi $ \s -> let (f', s') = f s
                                        (a, s'') = g s'
                                        a' = f' a
                                    in (a', s'')

instance Monad (Moi s) where
  return = pure

  (>>=) :: Moi s a -> (a -> Moi s b) -> Moi s b
  (Moi f) >>= g = Moi $ \s -> let (a, s')    = f s
                                  (a', s'') = (runMoi (g a)) s'
                              in (a', s'')
```

## Chapter 24 - Parser Combinators

- The core idea of parsing in programming is to accept serialized input in the form of a sequence of characters (textual data) or bytes (raw binary data) and turn that into a value of a structured datatype. Parsing breaks up a chunk of data and allows you to find and process the parts you care about.
- A *parser* is a function that takes some textual input and returns some *structure* as output. Parsers analyze structure in conformance with a grammar. A *parser combinator* is a higher-order function that takes parsers as input and returns a new parser as output.

- It's not a given that a single parser exhausts all of its input - they only consume as much text as they need to produce the value of the type requested.
- `<|>` from `import Control.Applicative` lets you combine two parsers to match this OR that. It is from the `Alternative` type class which also includes `some` (one or more) and `many` (zero or more) which can be used like `parseString (some letter) mempty "blah"` to return `"blah"`.
- Some other operators you commonly see in parsing are `>>`, `<*`, `*>`, `<$`, and `$>`. The first three are sequence operators. They execute both sides and keep the result on the side they are pointing to. The last two are the same except they point at a constant you keep the result of while executing the other side and throwing the value away, like `string "true" $> True`.
- When writing parsers in Haskell, it's often easier to work in terms of smaller parsers that deal with a sub-problem of the overall parsing problem you are solving, then combine them into the final parser. A good example can be found in `ch24/src/Data/Ini.hs` where we make an Ini file parser.
- Parser combinators are whitespace-sensitive by default, so without handling it explicitly, something like "1 2 3" won't parse as three numbers, the parser chokes on the space after "1". `token p` runs parser `p`, then automatically consumes and discards any trailing whitespace, letting token-wrapped parsers be chained together (e.g. `some (token digits)`) without manually skipping whitespace between them. Some parsers, like `integer`, are already tokenized for you, no need to wrap them again. Don't get tokenization happy, though: keep it coarse-grained and selective. Overusing tokenizing parsers can make things slow or hard to understand (e.g. don't double-wrap something already tokenized, like `token (some (token digits))`).
- You can annotate parses with `<?>`.

```
tryAnnot :: (Monad f, CharParsing f) => f Char
tryAnnot = (try (char '1' >> char '2') <?> "Tried 12" )
       <|> (char '3' <?> "Tried 3")

Prelude> trifP tryAnnot "13"
Failure (interactive):1:1: error: expected:
  Tried 12, Tried 3
13<EOF>
```

- Definitions

    1. A *parser* parses.
    2. A *parser combinator* combines two or more parsers to produce a new parser. A good example of this is using `<|>` from `Alternative` to produce a new parser from the disjunction of two parser arguments to `<|>`. Or `some`. Or `many`. Or `mappend`. Or `>>`.
    3. *Marshalling* is transforming a potentially nonlinear representation of data in memory into a format that can be stored on disk or transmitted over a network socket. Going in the opposite direction is called unmarshalling. Cf. *serialization* and *deserialization*.
    4. A *tokenizer* converts text, usually a stream of characters, into more meaningful or "chunkier" structures, such as words, sentences, or symbols - that is *tokens*. The `lines` and `words` functions you used earlier in this book ar unsophisticated tokenizers.
    5. *Lexer* - see *tokenizer*.

## Chapter 25 - Composing Types

- Functors and applicatives are both closed under composition. You can compose two functors (or two applicatives) and return another functor (or applicative). However, when you compose two monads, the result is not necessarily a monad. A monad transformer is a variant of any ordinary monad that takes an additional type argument that is assumed to have a `Monad` instance. For example, `MaybeT` is the transformer variants of the `Maybe` type. The transformer variant gives us a `Monad` instances that binds over both bits of structure allowing us to compose monads and combine their effects. Monad transformers are never sum or product types; they are always a means of wrapping one extra layer of (monadic) structure around another type, so they are often defined with `newtype`.

```haskell
newtype Identity a = Identity { runIdentity :: a }

newtype Compose f g a = Compose { getCompose :: f (g a) }
  deriving (Eq, Show)

ghci> Compose [Just 1, Nothing]
ghci> xs = [Just (1 :: Int), Nothing]
ghci> :t Compose xs
Compose xs :: Compose [] Maybe Int

instance Functor Identity where
  fmap f (Identity a) = Identity (f a)

instance (Functor f, Functor g) => Functor (Compose f g) where
  fmap f (Compose fga) = Compose $ (fmap . fmap) f fga

ghci> fmap (+1) (Compose xs)
Compose {getCompose = [Just 2, Nothing]}

instance (Applicative f, Applicative g) => Applicative (Compose f g) where
  pure :: a -> Compose f g a
  -- pure a = Compose (pure (pure a :: g a) :: f (g a))
  pure a = Compose (pure (pure a))

  (<*>) :: Compose f g (a -> b)
        -> Compose f g a
        -> Compose f g b
  -- (Compose f) <*> (Compose a) = Compose (liftA2 (<*>) f a)
  (Compose f) <*> (Compose a) = Compose ((<*>) <$> f <*> a)
```

- So Monads don't compose naturally. We need to know the type of one of the Monads to know how to join the two Monad binds. We'll start with `IdentityT`:

```haskell
newtype Identity a = Identity { runIdentity :: a }
  deriving (Eq, Show)

newtype IdentityT f a = IdentityT { runIdentityT :: f a }
  deriving (Eq, Show)

instance Functor Identity where
  fmap f (Identity a) = Identity (f a)

instance (Functor m) => Functor (IdentityT m) where
  fmap f (IdentityT fa) = IdentityT (fmap f fa)

instance Applicative Identity where
  pure = Identity
  (Identity f) <*> (Identity a) = Identity (f a)

instance (Applicative m) => Applicative (IdentityT m) where
  pure x = IdentityT (pure x)
  (IdentityT fab) <*> (IdentityT fa) = IdentityT (fab <*> fa)

instance Monad Identity where
  return = pure
  (Identity a) >>= f = f a

instance (Monad m) => Monad (IdentityT m) where
  return = pure
  (IdentityT ma) >>= f = IdentityT $ ma >>= runIdentityT . f
```

- As you see above, we need to know the `IdentityT` type concretely primarily so we know to call `runIdentityT` on the result of `ma >>= f` since `f` literally takes us past our desired result of `m b` and returns an `IdentityT m b` instead. `runIdentityT` unwraps that to the correct result for `ma`'s bind operation then we rewrap for the result of `IdentityT ma`'s bind operatation. The rewrap is a noop for `IdentityT` but could be important for say `StateT` or `ReaderT` (or `IO` but that's normally an inner monad in a stack and not a transformers).

- The book derived it a bit differently but had good general advice. Whenever you try to combine results and end up with an `m (T m b)` for some monads `m` and `T`, you need the transformer version of `T` so it can unwrap an intermediate result and you get an `m (m b)` which is easily handled by `join` to get an `m b` then rewrapped to a `T m b`.

## Chapter 26 - Monad Transformers

- `IdentityT` has its uses, but doesn't show up often, so we're moving on to `MaybeT`. You'll notice from the newtype that the actually `Maybe` goes in the inner layer and the base Monad goes on the outside.

```haskell
newtype MaybeT m a = MaybeT { runMaybeT :: m (Maybe a) }

instance (Functor m) => Functor (MaybeT m) where
  fmap f (MaybeT ma) = MaybeT $ (fmap . fmap) f ma

instance (Applicative m) => Applicative (MaybeT m) where
  pure x = MaybeT (pure (pure x))
  (MaybeT fab) <*> (MaybeT mma) = MaybeT $ (<*>) <$> fab <*> mma

instance (Monad m) => Monad (MaybeT m) where
  return = pure
  (>>=) :: MaybeT m a -> (a -> MaybeT m b) -> MaybeT m b
  (MaybeT ma) >>= f = MaybeT $ do
          -- ma :: m (Maybe a)
          -- v :: Maybe a
          v <- ma
          case v of
            Nothing -> return Nothing
            Just y -> runMaybeT (f y)
-- y :: a
-- f :: a -> MaybeT m b
-- f y :: MaybeT m b
-- runMaybeT (f y) :: m (Maybe b)
```

- We will also look at the useful `EitherT`:

```haskell
newtype EitherT e m a = EitherT { runEitherT :: m (Either e a) }

instance (Functor m) => Functor (EitherT m) where
  fmap f (EitherT ma) = EitherT $ (fmap . fmap) f ma

instance (Applicative m) => Applicative (EitherT e m) where
  pure x = EitherT (pure (pure x))
  (EitherT fab) <*> (EitherT mma) = EitherT $ (<*>) <$> fab <*> mma

instance (Monad m) => Monad (EitherT m) where
  return = pure
  (>==) :: EitherT m a -> (a -> EitherT m b) -> EitherT m b
  (EitherT ma) >>= f = EitherT $ do
          -- ma :: m (Either e a)
          -- v :: Either e a
          v <- ma
          case v of
            Left e -> return (Left e)
            Right y -> runEitherT (f y)

swapEither :: Either e a -> Either a e
swapEither (Left e) = Right e
swapEither (Right a) = Left a

swapEitherT :: (Functor m) => EitherT e m a -> EitherT a m e
swapEitherT (EitherT ma) = EitherT $ fmap swapEither ma

eitherT :: Monad m => (a -> m c) -> (b -> m c) -> EitherT a m b -> m c
eitherT f g (EitherT ma) = do $
          v <- ma
          case v of
            Left a -> f a
            Right b -> g b
```

- `ReaderT` is one of the most commonly used transfomers in conventional Haskell applications.

```haskell
newtype ReaderT r m a = ReaderT { runReaderT :: r -> m a }

instance (Functor m) => Functor (ReaderT r m) where
  fmap f (ReaderT rma) = ReaderT $ (fmap . fmap) f rma

instance (Applicative m) => Applicative (ReaderT r m) where
  pure a = ReaderT (pure (pure a))

  (ReaderT fmab) <*> (ReaderT rma) =
    ReaderT $ (<*>) <$> fmab <*> rma

instance (Monad m) => Monad (ReaderT r m) where
  return = pure

  (>>=) :: ReaderT r m a -> (a -> ReaderT r m b) -> ReaderT r m b
  (ReaderT rma) >>= f = ReaderT $ \r -> do
    a <- rma r
    runReaderT (f a) r
```

- `StateT` is `State` but with additional monadic structure wrapped around the result.

```haskell
newtype StateT s m a = StateT { runStateT :: s -> m (a, s) }

instance (Functor m) => Functor (StateT s m) where
  fmap f (StateT g) = StateT $ \s -> fmap (\(a, s') -> (f a, s')) (g s)
  -- or pointfree
  -- fmap f (StateT g) = StateT $ fmap (fmap (\(a, s') -> (f a, s'))) g

instance (Monad m) => Applicative (StateT s m) where
  pure x = StateT $ \s -> pure (x, s)

  (StateT smf ) <*> (StateT sma) = StateT $ \s -> do
    (f, s') <- smf s
    (a, s'') <- sma s'
    return (f a, s'')

instance (Monad m) => Monad (StateT s m) where
  return = pure

  (StateT sma) >>= f = StateT $ \s -> do
    (a, s') <- sma s
    runStateT (f a) s'
```

- You can use any monad transformer as the base monad by wrapping `Identity`.
- In general, don't roll your own transformers. There are well built versions of everything we discussed in the `transformers` library that comes packaged with GHC. Most of the time though we use `ExceptT` from `transformers` instead of `EitherT` which is defined in `either` on Hackage.
- In transformers, the Monad is wrapped around what we have but not what we need. This means either `Either` and `Maybe` are on the innermost layer in `ExceptT` and `MaybeT` because we have an `Either` or `Maybe` value inside the other monad, but when we have a `Reader`, it is `r -> m a` because we need an `r` so the `Reader` is the outer layer. When Haskellers refer to the *base* monad, they mean what is structurally outermost. If we declare `type MyType a = IO [Maybe a]` then the *base* is `IO`.
- Instances of `MonadTrans` provide a `lift` which raises functions to work in the context of the transformer stack. It works like an `fmap` or `liftM` that raises you the appropriate number of levels. If you find yourself chaining many calls of `lift`, it is a code smell. At the very least, write yourself an instance of `MonadTrans` for your stack so one `lift` is equal to `lift . lift . lift` for the stack.
- Any monad built by applying a sequence of manad transformers to the `IO` monad will be an instance of `MonadIO` which provides a `liftIO` that will lift an `IO` action until it is lifting over all structure embedded in the outermost `IO` type. This is from `Control.Monad.IO.Class`.

## Chapter 27 - Non-strictness

- Strict languages evaluate *inside out*; non-strict languages like Haskell evaluated *outside in*. The idea is that evaluation is driven by demand, not by constructor like a strict language.
- You can force evaulation with `seq`. `seq` evaluates its first argument when the second argument has to be evaluated. It isn't precisely like a strict langugae. It evanulates up to weak head normal form (the first data constructor or first lambda).
- If you `import Debug.Trace (trace)` you can use the `trace` function to see when things are evaluated. `trace` takes a string and an expression, and it prints the string when the expression is evaluated.
- You can make expressions be evaluated less by assigning them names. Haskell aggressively attempts to only evaluate a named expression once. Likewise, inlining values can cause them to be evaluated multiple times. You can also prevent sharing with a lambda

since Haskell doesn't memoize and cannot reuse function results. This can be valuable for a large datum that acts as an intermediate value that you don't want hanging out in memory. Note though that functions aren't shared when they have named arguments, but they are when written in point-free style.

- There are language extensions `Strict` and `StrictData` to make code strict. `StrictData` makes every field of every data type in the module strict (as if each field had a `!`). It's common, often enabled project-wide via `default-extensions`, and helps avoid space leaks from unevaluated thunks building up in long-lived records. `Strict` implies `StrictData` and also makes bindings and function arguments strict. It's heavy-handed and can change semantics (code relying on laziness can diverge), so you mostly see it in performance-sensitive modules. Either can be opted out of locally with `~`.record types.

## Chapter 28 - Basic Libraries

- When benchmarking compile your code with `-O` or `-O2` option, either by hand like `stack ghc -- -O2 bench.hs` or in `ghc-options`.
- Criterion is a good benchmarking package. When running benchmarks in `main` just `import Criterion.Main` and use `bench`.
- `stack ghc -- -prof -fprof-auto -rtsopts -O2 profile.hs` then running it with `./profile +RTS -P` will profile your code and generate a readable `'profile.prof`. See the GHC Docs for more info. You can also do memory profiling with `stack ghc -- -prof -fprof-auto -rtsopts -O2 loci.hs` then `./loci +RTS -hc -p` and calling `hp2ps loci.hp` and viewing the postscript output in a PDF reader. Criterion also has `whnf` and `nf` functions to partially evaluate expressions. Note that Criterion is no longer maintained by the original author but has been picked up by the community.
- For data structures we will mostly focus on the `containers` library. It is documented here. You're best using `Data.Map` anytime you have keys and values since it promises fast lookups. If your key type is `Int`, you might be better of with a `HashMap`, `IntMap`, or `Vector`. `Data.Set` is for unique, ordered lists of just values. Unlike traditional lists, `Data.Sequence` is a structure where you can append to either end quickly. `data.Vector` is in the `vector` library, Vector documentation. You want a `Vector` when you need memory efficiency, your data access is indexing via and `Int` value, you want uniform performance, and you will construct a `Vector` once and read it many times. `Data.Vector.Mutable` is a mutable Vector.
- `String` is good enough for demonstrations or toy programs. `Text` from the `text` library is much better for memory usage and efficient indexing into the string. `ByteString` is also

used for `String` values but is a `Vector` of `Word8` byte values. It is in the `bytestring` library. `Char8` is not for Unicode or more generally for text.

## Chapter 29 - IO

- `IO` primarily exists to give us a way to order operation and to disable some of the sharing we saw in Chapter 27. A value of `IO a` isn't an `a` but a description of how you might get an `a`.

## Chapter 30 - When Things Go Wrong

- An exception is a type with an instance of the `Exception` type class.

```
class (Typeable e, Show e) => Exception e where
  toException :: e -> SomeException
  fromException :: SomeException -> Maybe e
  displayException :: e -> String

data SomeException where
  SomeException :: Exception e => e -> SomeException
```

- We don't use `toException` and `fromException` directly and instead call functions that call them for us. Any type that implements the `Exception` class can be that `e` and be subsumed under the `SomeException` type.

- The `Typeable` type class lives in the `Data.Typable` module. You do not need to explicitly derive `Typeable` on your datatypes in order to use the `Data.Typable` API.

- At runtime when an exception is thrown it starts rolling back through the call stack looking for a `catch`. When it finds a `catch` it checks to see what type of exception this `catch` catches. A `catch` that handles `SomeException` will match any type of exception. Youc an only handle exceptions in `IO`, because `IO` comes with the implicit contract, "You cannot expect this computation to succeed unconditionally." If nothing catches an exception, it kills the program.

```
catch :: Exception e => IO a -> (e -> IO a) -> IO a

-- Control.Exception
try :: Exception e => IO a -> IO (Either e a)
```

- `throwIO` allows you to throw an exception. There is also `throw` to throw exceptions without IO. You almost never want `thrown`,
- Note that `Exception` instances are derivable by `instance Exception Type`. If your type takes an argument, it is included as extra information.

## Chapter 31 - Final Project

- We are going to write a finger server, but first a little `Debug.hs` program to display working with network sockets by echoing inputs:
- Note the `network` library has changed since the book is published, so this doesn't match the book.

```haskell
module Main where

import Control.Monad (forever)
import Network.Socket hiding (recv)
import Network.Socket.ByteString (recv, sendAll)

logAndEcho :: Socket -> IO ()
logAndEcho sock = forever $ do
  (soc, _) <- accept sock
  printAndKickback soc
  close soc

  where printAndKickback conn = do
          msg <- recv conn 1024
          print msg
          sendAll conn msg

main :: IO ()
main = withSocketsDo $ do
  addrinfos <- getAddrInfo
               (Just (defaultHints
                 {addrFlags =
                  [AI_PASSIVE]}))
               Nothing (Just "79")

  let serveraddr = head addrinfos
  sock <- socket (addrFamily serveraddr)
                 Stream defaultProtocol

  bind sock (addrAddress serveraddr)
  listen sock 1
  logAndEcho sock
  close sock
```

- First, let's analyze `logAndEcho`. We are using `forever` to keep the socket open indefinitely. The `accept` command blocks until a client connects to the server, and `soc` is the result of `accept`-ing a connection for communicating with the client. The server will receive up to 1024 bytes of text from the client, print the text literally, then echo the text back to the client across the connection. We close `soc` to close the active connection but not `sock` since we still want to listen for other connections on the server socket. Because we're looping with `forever`, the next action after closing the socket is to block and wait for another connection to `accept`.

- Now let's look at `main`. `withSocketsDo` does nothing except on Windows where it is required to use the socket API. The `addrinfos` is mostly ceremony but the `Just "79"` portion is the port we are listening on. Note that we'll need to run as Administrator or root to open a server of port 79. The next bit uses `socket` to bind to the address and port we specified. Then we `listen` on the socket we are bound to. If `logAndEcho` finishes (remember, it's running forever), we close the socket and exit.
- We can run our echo server with `sudo `stack exec which debug``.
- The code is pretty self explanatory, so I'll just present the entire `Main.h` file for `fingerd` (even though you could view it in the repository).

```haskell
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE QuasiQuotes       #-}
{-# LANGUAGE RecordWildCards   #-}
module Main (main) where

import Control.Exception
import Control.Monad (forever)
import Data.List (intersperse)

import Data.Text(Text)
import qualified Data.Text as T
import Data.Text.Encoding (decodeUtf8, encodeUtf8)
import Data.Typeable
import Database.SQLite.Simple hiding (bind, close)
import qualified Database.SQLite.Simple as SQLite

import Database.SQLite.Simple.Types
import Network.Socket hiding (recv)
import Data.ByteString (ByteString)

import qualified Data.ByteString as BS
import Network.Socket.ByteString (recv, sendAll)
import Text.RawString.QQ

data User =
  User {
      userId   :: Integer
    , username :: Text
    , shell :: Text

    , homeDirectory :: Text
    , realName :: Text
    , phone :: Text
  } deriving (Eq, Show)

instance FromRow User where
  fromRow = User <$> field
                 <*> field
                 <*> field
                 <*> field
                 <*> field
                 <*> field

instance ToRow User where
  toRow (User id_ username shell homeDir realName phone) =
```

```
    toRow (id_, username, shell, homeDir, realName, phone)

createUsers :: Query
createUsers = [r|
CREATE TABLE IF NOT EXISTS users
  (id INTEGER PRIMARY KEY AUTOINCREMENT,
   username TEXT UNIQUE,
   shell TEXT, homeDirectory TEXT,
   realName TEXT, phone TEXT)
|]

insertUser :: Query
insertUser =
  "INSERT INTO users\
  \ VALUES (?, ?, ?, ?, ?, ?)"

allUsers :: Query
allUsers =
  "SELECT * from users"

getUserQuery :: Query
getUserQuery =
  "SELECT * from users where username = ?"

data DuplicateData = DuplicateData
  deriving (Eq, Show, Typeable)

instance Exception DuplicateData

type UserRow = (Null, Text, Text, Text, Text, Text)

getUser :: Connection -> Text -> IO (Maybe User)
getUser conn username = do
  results <- query conn getUserQuery (Only username)
  case results of
    [] -> return $ Nothing
    [user] -> return $ Just user
    _ -> throwIO DuplicateData

createDatabase :: IO ()
createDatabase = do
  conn <- open "finger.db"
  execute_ conn createUsers
  execute conn insertUser meRow

  rows <- query_ conn allUsers
```

```
  mapM_ print (rows :: [User])
  SQLite.close conn

  where meRow :: UserRow
        meRow = (Null, "callen", "/bin/zsh",
                 "/home/callen", "Chris Allen", "555-123-4567")

returnUsers :: Connection -> Socket -> IO ()
returnUsers dbConn soc = do
  rows <- query_ dbConn allUsers

  let usernames = map username rows
      newlineSeparated = T.concat $ intersperse "\n" usernames

  sendAll soc (encodeUtf8 newlineSeparated)

formatUser :: User -> ByteString
formatUser (User _ username shell homeDir realName _) = BS.concat

  ["Login: ", e username, "\t\t\t\t",
   "Name: ", e realName, "\n",
   "Directory: ", e homeDir, "\t\t\t",
   "Shell: ", e shell, "\n"]
  where e = encodeUtf8

returnUser :: Connection -> Socket -> Text -> IO ()
returnUser dbConn soc username = do
  -- the literal `username` text ends with `\r\n` so we strip it
  maybeUser <- getUser dbConn (T.strip username)

  case maybeUser of
    Nothing -> do
      putStrLn ("Couldn't find matching user\
                \ for username: " ++ (show username))
      return ()

    Just user -> sendAll soc (formatUser user)

handleQuery :: Connection -> Socket -> IO ()
handleQuery dbConn soc = do
  msg <- recv soc 1024

  case msg of
    "\r\n" -> returnUsers dbConn soc
    name -> returnUser dbConn soc (decodeUtf8 name)
```

```haskell
handleQueries :: Connection -> Socket -> IO ()
handleQueries dbConn sock = forever $ do

  (soc, _) <- accept sock
  putStrLn "Got connection, handling query"

  handleQuery dbConn soc
  close soc

main :: IO ()
main = withSocketsDo $ do
  addrinfos <-
    getAddrInfo
    (Just (defaultHints
      {addrFlags = [AI_PASSIVE]}))
    Nothing (Just "79")

  let serveraddr = head addrinfos
  sock <- socket (addrFamily serveraddr)
          Stream defaultProtocol
  bind sock (addrAddress serveraddr)

  listen sock 1
  -- only one connection open at a time
  conn <- open "finger.db"
  handleQueries conn sock

  SQLite.close conn
  close sock
```

- Note that you need to run `createDatabase` by hand. You can run a shell in the context of this program with `stack ghci --main-is fingerd:exe:fingerd`. `stack build` will build everything, and again we need to run this with `sudo` like `sudo \`stack exec which fingerd\`` because it binds to a low port.